

BTS CIEL option A
Cybersécurité, l'informatique et réseaux et l'électronique
E6 – PROJET INFORMATIQUE
Dossier de réalisation du projet.

Académies : Besançon-Dijon	Session : 2026
Lycée ou Centre de formation : Gustave Eiffel	
Ville : Dijon	
Nom du projet : Séchoir a houblon	
Projet industriel : OUI NON	

Équipe de développement.	
Professeur :	<i>Raoul Artola</i>
L'entreprise :	<i>Adrien Darphin</i>
Interlocuteur(s) de l'entreprise.	<i>Adrien Darphin (agriculteur)</i>
Étudiant 1 :	<i>Lénny Sauvage</i>
Étudiant 2 :	<i>Adam Cardoso</i>
Étudiant 3 :	<i>Ayoub Rabhi</i>

Table des matières

1. Présentation du projet.....	5
1.1 Problématique du projet.....	5
1.2 Expression du besoin.....	5
1.3 Rôles par étudiant.....	5
1.4 Structure matérielle et logicielle du système.....	5
2. Analyse et spécifications.....	6
2.1 Diagramme de déploiement.....	6
2.2 Diagramme de cas d'utilisation générale.....	6
2.3 Expression fonctionnelle du besoin.....	7
2.4 Diagramme d'exigences — RTC et base de données.....	8
2.5 Diagramme d'exigences — Service web.....	8
2.6 Diagramme de séquence.....	10
2.7 Évolutions et adaptations en cours de développement.....	11
2.8 Tests réalisés sur le système.....	11
3. Méthodologie de prise en compte des contraintes du cahier des charges.....	11
4. Contenu du support de rendu.....	11
Présentation du travail du candidat N°1 – Lénny Sauvage.....	12
Introduction.....	12
5. Configuration du service web (Apache 2 + PHP).....	12
5.1 Contexte et objectif.....	12
5.2 Choix techniques.....	12
5.3 Périmètre de ma réalisation.....	12
5.4 Installation et configuration.....	12
5.4.1 Installation d'Apache 2.....	12
5.4.2 Installation de PHP 8.4.....	13
5.4.3 Organisation du site.....	13
5.4.4 Droits d'accès (principe de moindre privilège).....	13
5.4.5 Configuration du VirtualHost.....	13
5.5 Fonctionnement des échanges front-end / back-end.....	13
6. Configuration du service base de données (MariaDB).....	14
6.1 Contexte et objectif.....	14
6.2 Périmètre de ma réalisation.....	14
6.3 Architecture de la base.....	14
6.4 Modèle Conceptuel de Données (MCD).....	14
6.5 Modèle Logique de Données (MLD).....	15
6.6 Ajout d'une variété.....	16
7. Configuration de l'horloge temps réel (RTC).....	16
7.1 Contexte et objectif.....	16
7.2 Solution retenue.....	16
7.3 Périmètre de ma réalisation.....	16
7.4 Étude comparative des modules RTC externes.....	16
7.5 Analyse des risques liés à l'absence de RTC.....	17
7.6 Procédure de configuration — Raspberry Pi 5 (RTC interne).....	17
Étape 1 — Connexion de la pile.....	17
Étape 2 — Réglage initial de l'heure.....	17
Étape 3 — Vérification du fonctionnement.....	17
Étape 4 — Vérification finale après coupure.....	17
7.7 Bilan.....	18
8. Développement de l'IHM de configuration du houblon.....	18
8.1 Contexte et objectif.....	18
8.2 Fonctionnalités développées.....	18
8.3 Périmètre de ma réalisation.....	18
8.4 Architecture technique de l'IHM.....	18

8.4.1 Structure des fichiers.....	18
8.4.2 Architecture JavaScript modulaire.....	18
8.5 Comportement de l'overlay selon le contexte.....	19
8.6 Communication avec l'API.....	20
8.7 Mise à jour automatique de l'affichage.....	22
8.8 Difficultés rencontrées et solutions apportées.....	22
9. Développement de l'IHM d'export de données.....	23
9.1 Contexte et objectif.....	23
9.2 Fonctionnalités développées.....	23
9.3 Périmètre de ma réalisation.....	23
9.4 Architecture technique de la page.....	23
9.4.1 Parcours utilisateur en trois étapes.....	23
9.4.2 Chargement dynamique des données de filtrage.....	23
9.4.3 Filtrage et prévisualisation des résultats.....	23
9.4.4 Génération et téléchargement du fichier CSV.....	23
9.5 Difficultés rencontrées et solutions apportées.....	24
Conclusion.....	24
Présentation du travail du candidat N°2 – Ayoub Rabhi.....	25
Prototypage.....	25
Phase 1 : Validation de la mesure et de la logique d'alerte.....	25
Phase 2 : Prototypage de l'architecture Multi-Bus.....	26
Liste des objets :.....	29
Diagrammes de cas d'utilisations:.....	29
Diagrammes de déploiement :.....	31
Diagrammes d'exigences :.....	31
Diagrammes de séquences entre objet :.....	33
Conception détaillée.....	38
Étude du protocole et de la source de données.....	38
Algorithme de la lecture fiabilisée (Méthode ' read_temp_fiable').....	38
Pour garantir la précision demandée au cahier des charges, j'ai conçu un algorithme basé sur un cycle d'échantillonnage de 5 mesures.....	38
Plan de test unitaire de la lecture fiabilisée (Méthode ' read_temp_fiable') :.....	41
Algorithme de surveillance et connectivité :.....	41
Plan de test unitaire de surveillance et connectivité :.....	41
Codage.....	42
Les tests unitaires :.....	42
Bilan personnel :.....	43
Planning réalisé :.....	43
Difficultés rencontrées :.....	43
Présentation du travail du candidat N°3 – Adam Cardoso.....	45
Introduction.....	45
Présentation de ma mission individuellement.....	45
Déroulement des tâches réalisées.....	45
1. Conception du site web « Contrôler le séchage » - Structure HTML.....	45
1.1 Mise en forme et design – CSS.....	46
1.2 Développement PHP.....	47
1.3 Affichage des températures.....	47
1.4 Gestion de la pause et de la reprise du séchoir.....	47
2. Configuration du point d'accès Wi-Fi.....	50
2.1 Objectif de la configuration.....	50
2.2 Installations des services nécessaires.....	50
3.3 Configuration du service hostapd.....	50
Création du fichier de configuration.....	50
Déclaration du fichier de configuration.....	51

3.4 Attribution d'une adresse IP statique.....	51
3.5 Configuration du service dnsmasq.....	51
Sauvegarde de l'ancienne configuration.....	51
Création de la nouvelle configuration.....	51
3.6 Gestion de NetworkManager.....	52
3.7 Démarrage du point d'accès.....	52
3.8 Accès au serveur web depuis un smartphone.....	52
3.9 Problèmes rencontrés et solutions apportées.....	52
3.91 Résultat final.....	52
4. Conception de la page « Ajouter masse houblon ».....	53
4.1 Objectif de la page.....	53
4.2 Architecture générale de la page.....	53
4.3 Interface utilisateur.....	53
Sélection des lots.....	53
Saisie de la masse.....	54
Interaction et validation des données.....	54
Système de confirmation.....	54
Envoi des données au serveur.....	54
4.4 Traitement côté serveur (PHP).....	55
Validation des données.....	55
4.5 Insertion en base de données.....	55
4.6 Gestion des réponses serveur.....	55
4.7 Réinitialisation de l'interface.....	56
4.8 Bilan de la conception.....	56
5. Choix et mise en place du voyant lumineux.....	56
5.1 Choix du voyant.....	56

1. Présentation du projet

1.1 Problématique du projet

Le séchage du houblon est une étape critique qui doit être réalisée dans les 4 à 6 heures suivant la récolte, à une température maintenue entre 50 et 55°C, sans dépasser 60°C. Actuellement, ce processus est assuré manuellement, obligeant le propriétaire à rester en permanence sur place pour surveiller l'état du houblon. Cette contrainte est à la fois chronophage, fatigante et source d'erreurs humaines. Il devient donc nécessaire d'automatiser le contrôle du séchage afin de libérer l'exploitant tout en garantissant la qualité du produit et en optimisant la consommation énergétique.

1.2 Expression du besoin

Le système doit permettre à l'exploitant de piloter et surveiller à distance, depuis son smartphone, l'intégralité du cycle de séchage du houblon. Il doit automatiser la régulation de la température via le contrôle du brûleur à gaz et du ventilateur, mesurer en continu la température du séchoir, déclencher des alertes sonores et visuelles en cas de dépassement de seuil ou de fin de cycle, et enregistrer l'historique complet des données dans une base de données. L'interface web doit également permettre la saisie des paramètres de séchage, le suivi des variétés de houblon produites et l'export des données en fichier CSV sur clé USB.

1.3 Rôles par étudiant

Étudiant 1 — Interface utilisateur, RTC et base de données : il est responsable de la mise en œuvre de l'horloge temps réel (RTC), de l'installation du service web (Apache) et de la configuration de la base de données (MariaDB), de la conception de l'interface de saisie des paramètres utilisateur (variété, températures min/max, durée de séchage) ainsi que de la visualisation des données historiques sous forme de tableaux et graphiques. Il développe également la fonctionnalité d'export CSV.

Étudiant 2 — Acquisition des données et régulation thermique : il prend en charge le choix et la mise en œuvre des 6 capteurs de température, l'interfaçage électronique avec la Raspberry Pi, et le développement du programme de régulation tout ou rien du brûleur à gaz. Il gère également le déclenchement de l'alerte sonore (sirène) en cas de dépassement des 60°C, ainsi que l'enregistrement horodaté des mesures, commandes et alertes dans la base de données.

Étudiant 3 — Contrôle du séchage, alerte visuelle et connectivité WiFi : il est responsable de la configuration du point d'accès WiFi, du développement de la page web de contrôle du séchage (visualisation des températures, état du chauffage et de la ventilation, temps restant, alertes), de la mise en œuvre du voyant d'alerte visuelle de fin de cycle, et des fonctionnalités d'ajout de la masse de houblon produit. Il gère aussi l'enregistrement en base de données des événements de démarrage et d'arrêt des cycles.

1.4 Structure matérielle et logicielle du système

Matériel : le système repose sur une Raspberry Pi 5 comme contrôleur central, 6 capteurs de température étanches connectés en série, une RTC interne, des interfaces de puissance TOR commandant le brûleur à gaz, le ventilateur triphasé, la sirène et le voyant lumineux. L'armoire

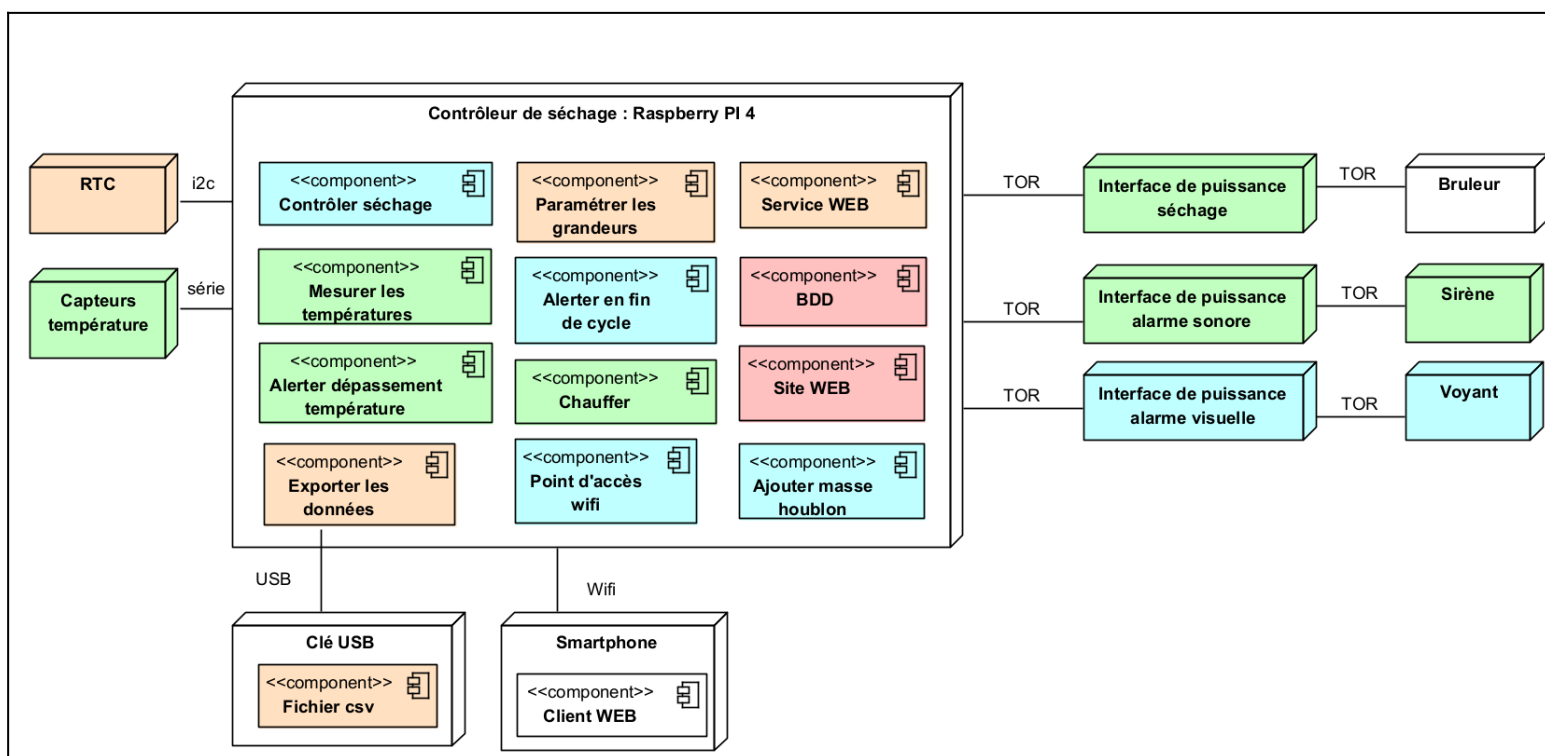
électrique est pré-câblée avec boutons Marche/Arrêt. Le smartphone sert de terminal de supervision via WiFi.

Logiciel : Raspberry Pi OS 64 bits, serveur web Apache 2, PHP 8.4, base de données MariaDB, développement en HTML5, CSS3, JavaScript, PHP et C. L'interface utilisateur est un site web responsive accessible depuis un navigateur mobile, sans installation côté client.

2. Analyse et spécifications

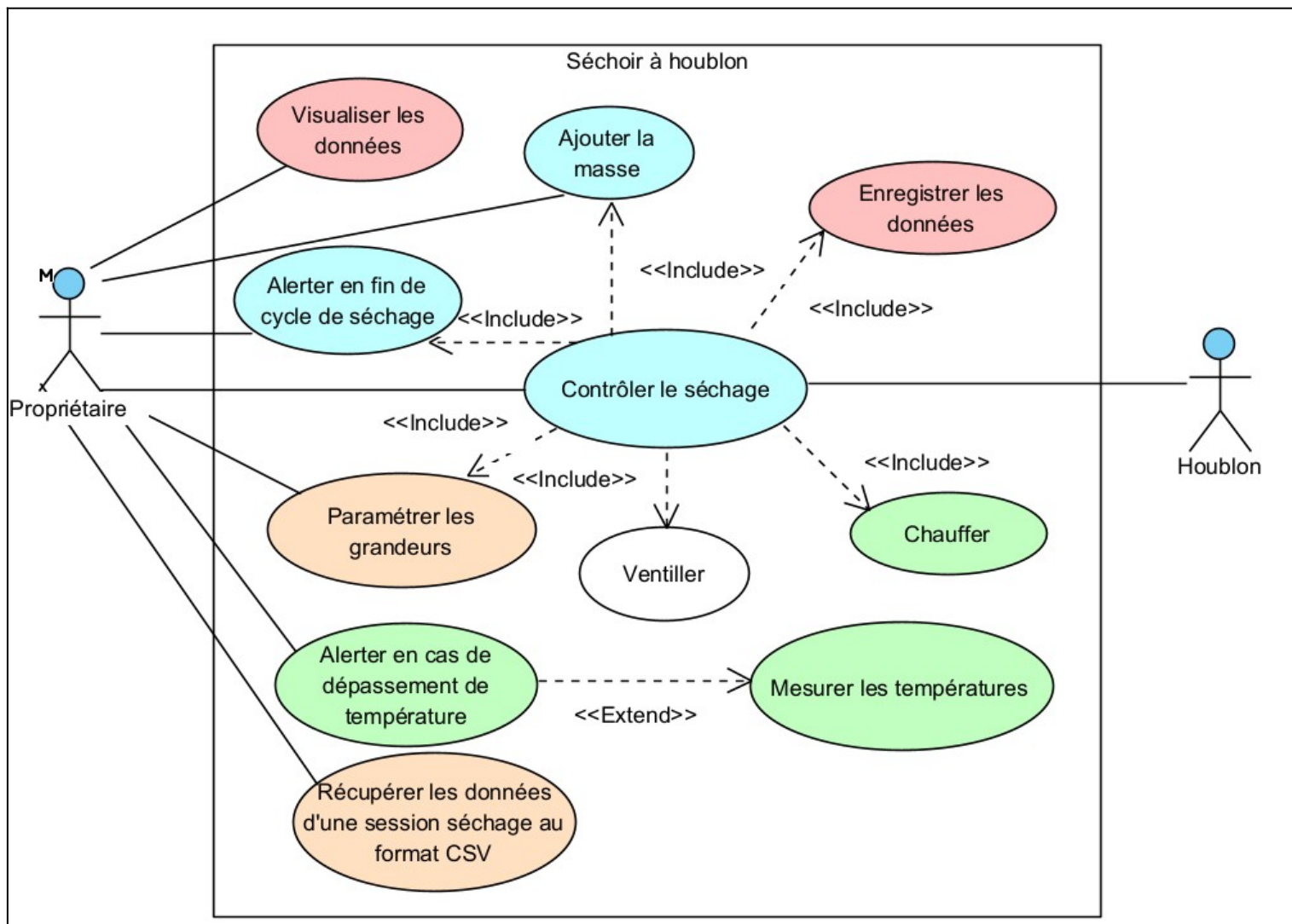
2.1 Diagramme de déploiement

Le système s'articule autour de la Raspberry Pi 5, qui constitue le nœud central de l'architecture. Côté matériel, elle reçoit les données des 6 capteurs de température via une liaison série, synchronise l'heure grâce à sa RTC interne, et pilote les trois interfaces de puissance TOR commandant respectivement le brûleur de séchage, la sirène sonore et le voyant lumineux. Côté logiciel, cette même Raspberry Pi héberge l'ensemble de la pile applicative : le service web Apache, le moteur PHP 8.4 et le serveur de base de données MariaDB. Le smartphone de l'utilisateur se connecte en WiFi et accède au site web pour superviser et contrôler le séchage en temps réel.



2.2 Diagramme de cas d'utilisation générale

Le système de séchage automatisé a été conçu pour répondre à un besoin concret : permettre au propriétaire de piloter et surveiller son séchoir à houblon à distance, sans nécessiter sa présence permanente sur les lieux. Le propriétaire peut ainsi configurer les paramètres de séchage, superviser en temps réel l'état du système, être alerté en cas d'anomalie ou de fin de cycle, et consulter l'historique des sessions passées.



2.3 Expression fonctionnelle du besoin

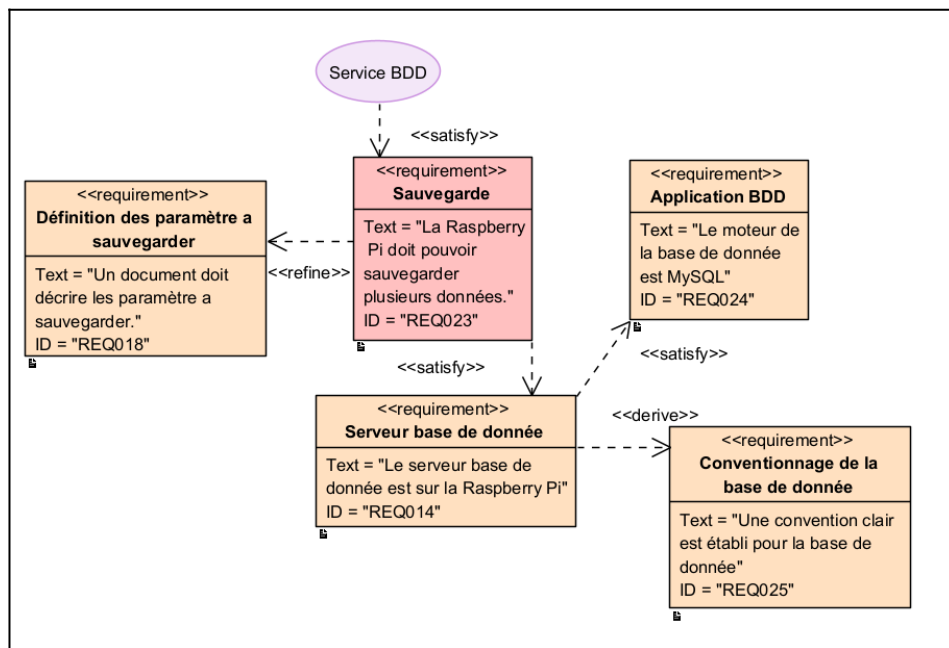
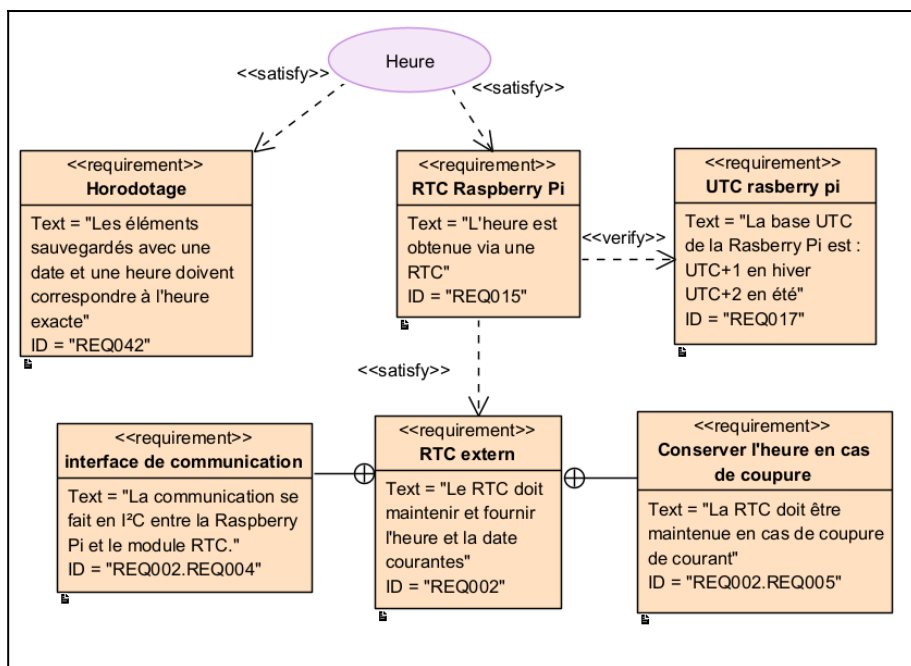
L'utilisateur principal du système est l'exploitant. Il interagit avec le système depuis son téléphone via un navigateur web connecté au point d'accès WiFi local du Raspberry Pi. Le système doit être simple d'utilisation, robuste et adapté à un environnement agricole.

Fonction	Description attendue	Utilisateur concerné
Paramétrer un cycle	Saisie variété, températures min/max et durée de séchage.	Exploitant
Contrôler le séchage	Visualisation des 6 températures, moyenne, état chauffage/ventilation, temps restant.	Exploitant
Mesurer	Mesures toutes les minutes sur 6 capteurs.	Système
Chauffer	Commande tout ou rien du brûleur selon les seuils.	Système
Alerter	Sirène en cas de dépassement et voyant en fin de cycle.	Système
Enregistrer	Historique horodaté des mesures, actions et alertes.	Système
Exporter	Création d'un fichier CSV sur clé USB.	Exploitant

L'expression du besoin a été traduite en fonctions testables afin de faciliter la recette finale du projet.

2.4 Diagramme d'exigences — RTC et base de données

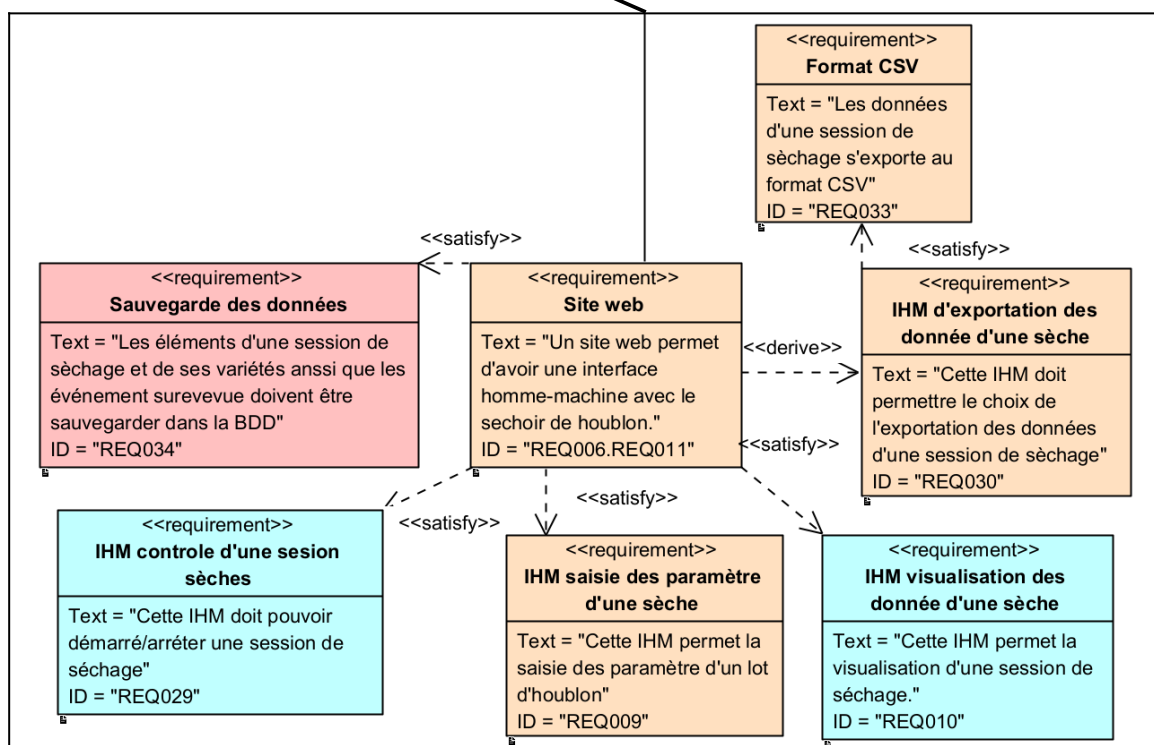
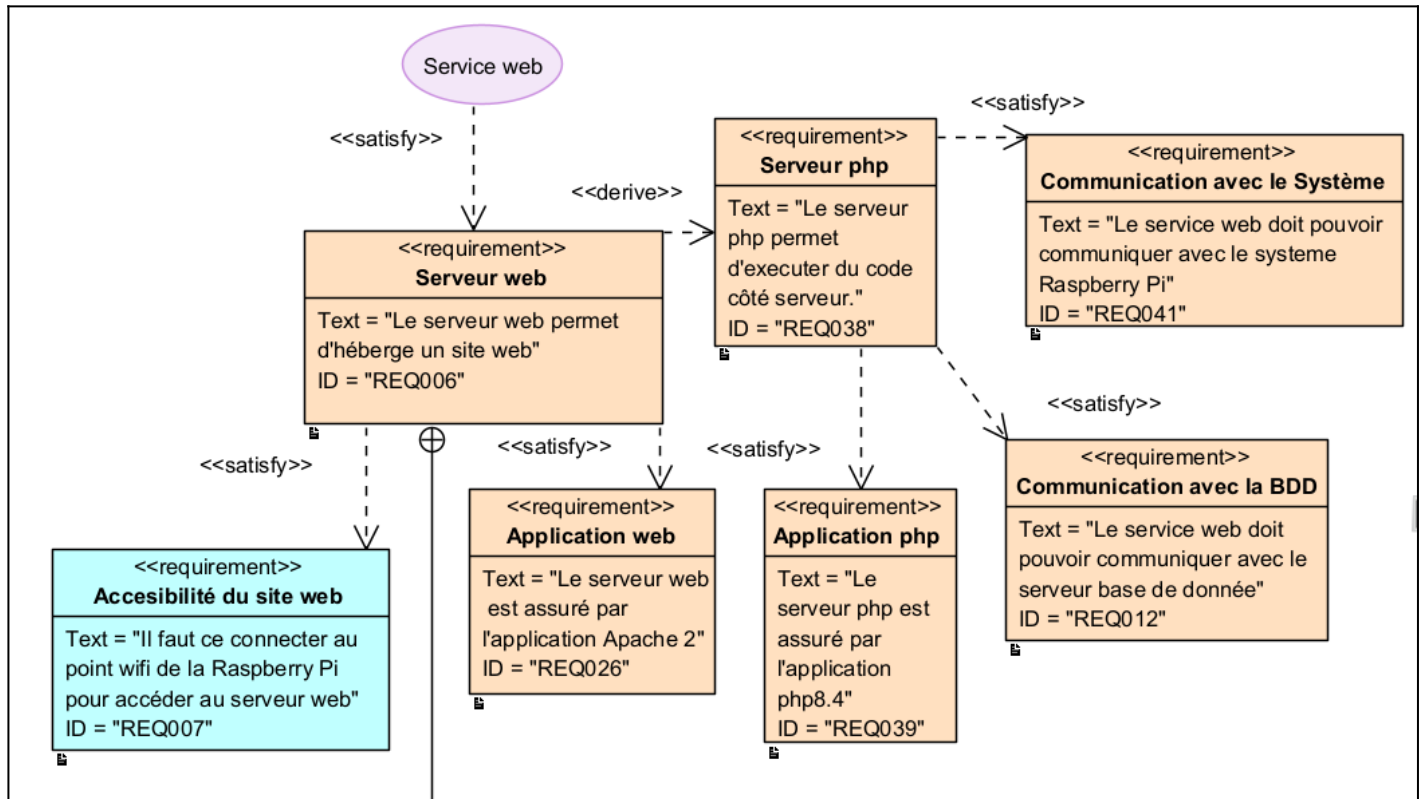
Deux composants sont au cœur de la fiabilité du système : l'horloge temps réel (RTC) et la base de données. La RTC est indispensable pour garantir un horodatage précis de toutes les actions et mesures enregistrées tout au long du cycle de séchage. Elle maintient l'heure même en cas de coupure de courant. Le fuseau horaire est configuré en Europe/Paris. La base de données MariaDB est hébergée directement sur la Raspberry Pi et centralise les mesures de température, les événements de chauffage, les pauses et événements, et les informations liées aux variétés de houblon produites



2.5 Diagramme d'exigences — Service web

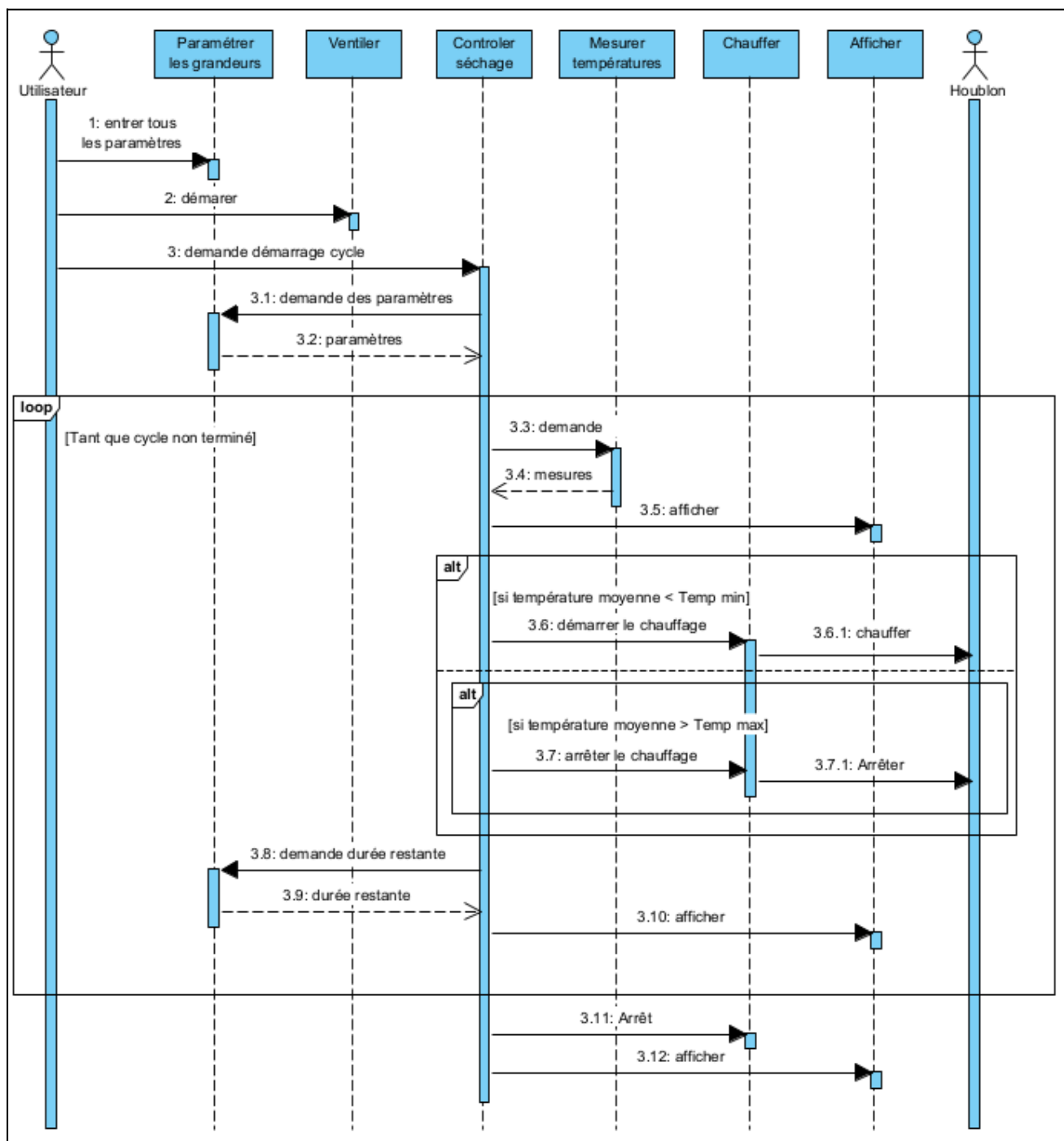
L'interface utilisateur repose entièrement sur un service web hébergé sur la Raspberry Pi, accessible depuis le smartphone du propriétaire via le point d'accès WiFi dédié. Ce service est composé d'un serveur Apache 2 et d'un serveur PHP 8.4. Le service web communique avec la Raspberry Pi pour

recupérer l'état du système en temps réel, et avec la base de données pour lire et écrire les informations liées aux cycles de séchage.



2.6 Diagramme de séquence

Le cycle de séchage débute par la saisie des paramètres par l'utilisateur : variété de houblon, températures minimale et maximale, et durée de séchage. Une fois ces paramètres validés, l'utilisateur démarre la ventilation depuis l'armoire électrique, ce qui lance officiellement le cycle. Le système entre alors dans une boucle de fonctionnement continu : les températures sont mesurées régulièrement et affichées sur l'interface web. Si la température moyenne descend en dessous du seuil minimal, le brûleur est automatiquement allumé. À l'inverse, si elle dépasse le seuil maximal, le brûleur est immédiatement coupé. En parallèle, le temps restant avant la fin du cycle est calculé et affiché en permanence. Lorsque la durée de séchage est écoulée, le système arrête le chauffage et en informe l'utilisateur via une alerte visuelle.



2.7 Évolutions et adaptations en cours de développement

Au cours du projet, plusieurs adaptations ont été nécessaires par rapport au cahier des charges initial. La principale est le remplacement de la Raspberry Pi 3 initialement prévue par une Raspberry Pi 5, fournie en cours de développement. Ce changement a eu des impacts directs : la Raspberry Pi 5 intègre nativement une RTC interne, supprimant le besoin d'un module I2C externe. La version du serveur PHP a également évolué vers PHP 8.4, et le serveur de base de données MySQL a été remplacé par MariaDB, compatible mais plus adapté à Raspberry Pi OS.

2.8 Tests réalisés sur le système

Des tests ont été menés de manière incrémentale, en validant chaque composant individuellement avant intégration dans le système complet. Chaque fonctionnalité développée a fait l'objet de tests unitaires documentés, couvrant les cas nominaux et les cas limites (dépassement de température, perte de connexion, données manquantes en base).

3. Méthodologie de prise en compte des contraintes du cahier des charges

La méthodologie adoptée tout au long du projet repose sur une lecture attentive du cahier des charges en amont de chaque développement, afin d'identifier les contraintes techniques et fonctionnelles à respecter avant toute implémentation. Les choix techniques ont été systématiquement justifiés et documentés.

Le travail a été organisé selon les rôles définis dans le cahier des charges, avec des points de synchronisation réguliers entre les trois étudiants afin d'assurer la cohérence de l'architecture globale, notamment au niveau de l'interface et/ou entre les scripts PHP de chaque étudiant et aussi pour la structure de la base de données.

4. Contenu du support de rendu

Le support de rendu est une clé USB qui doit contenir : ce rapport, les codes sources (dernière version), ainsi que les manuels d'utilisation et d'installation du système. La hiérarchie est la suivante :

- Répertoire src/ : sources de tous les programmes.
- Répertoire bin/ : exécutable.
- Répertoire doc/ : documentations.
- Répertoire installation/ : fichiers d'installation de l'application.

Le projet étant coordonné via git, un repo disponible en ligne sur github est accessible via l'url → <https://github.com/Sauvage89/sechoir-houblon>

La navigation au sein de ce repo est documenter dans le **README.md** qui se trouve à la racine du projet.

Présentation du travail du candidat N°1 – Lénny Sauvage

Introduction

Dans le cadre du projet Séchoir à Houblon, ma contribution repose principalement sur deux axes : la mise en place de l'infrastructure système de la Raspberry Pi (serveur web Apache, base de données MariaDB, horloge temps réel), et le développement des deux interfaces utilisateur qui permettent à l'agriculteur de configurer les lots d'houblon et d'exporter l'historique de sa production.

Ce dossier présente, pour chacun de ces cinq sujets, le contexte et l'objectif, les choix techniques retenus, le périmètre exact de ma réalisation, ainsi que les difficultés rencontrées et les solutions apportées.

Tout test est vérifié par un cahier de recette.

Toute documentation technique se situe dans le dossier guide/ dans la clé USB ou bien dans le dossier doc/guide/ du github du projet.

5. Configuration du service web (Apache 2 + PHP)

5.1 Contexte et objectif

L'agriculteur doit pouvoir interagir avec le système de contrôle depuis son smartphone, sans aucune installation logicielle particulière. Le choix d'une interface web s'est imposé comme solution naturelle : accessible depuis n'importe quel navigateur, elle ne nécessite aucun déploiement côté client et reste simple à maintenir.

Le serveur web est hébergé sur la Raspberry Pi 5 et constitue le point d'entrée unique pour la consultation des températures, le paramétrage du séchage, le pilotage du cycle (démarrage/arrêt), ainsi que la visualisation et l'export des données.

5.2 Choix techniques

L'environnement technique retenu repose sur les technologies imposées par le cahier des charges :

- Apache 2 : serveur HTTP, choisi pour sa robustesse, sa disponibilité sur Raspberry Pi OS et sa large documentation.
- PHP 8.4 : langage serveur permettant l'exécution de scripts et l'accès à la base de données.
- HTML5 / CSS3 / JavaScript / C : couche présentation côté navigateur, avec appels asynchrones (fetch) vers les scripts PHP.

5.3 Périmètre de ma réalisation

Ma contribution a porté sur l'installation et la configuration complète du serveur Apache 2 sur la Raspberry Pi, l'activation du module PHP, la définition des droits d'accès selon le principe de moindre privilège (utilisateur www-data), ainsi que la mise en place de la structure de fichiers du site web. La configuration du VirtualHost (fichier 000-default.conf) et la politique de droits sur les répertoires /var/www/html et /var/log/apache2 ont été définies et appliquées.

5.4 Installation et configuration

5.4.1 Installation d'Apache 2

L'installation du serveur web a été réalisée via le gestionnaire de paquets APT. Une vérification du bon fonctionnement a été effectuée en accédant à la page par défaut du serveur depuis un navigateur web sur le réseau local. Le service Apache a ensuite été configuré pour démarrer automatiquement au démarrage du système.

5.4.2 Installation de PHP 8.4

Le module PHP a été installé et intégré à Apache afin de permettre l'exécution de scripts côté serveur. Cette étape est essentielle car l'ensemble de la logique métier (traitement des données, interaction avec la base) repose sur PHP.

5.4.3 Organisation du site

Le site est déployé dans `/var/www/html`, avec la structure suivante :

- `api/` : scripts PHP (traitement, accès BDD)
- `site/` : pages HTML et PHP
- `css/` : feuilles de style (une par page)
- `js/` : scripts Javascript

Cette organisation permet une séparation claire entre front-end et back-end, facilitant la maintenance et le travail collaboratif.

5.4.4 Droits d'accès (principe de moindre privilège)

Le processus Apache s'exécute sous l'utilisateur système `www-data`, qui ne dispose que des droits strictement nécessaires :

- `/var/www/html` et ses sous-dossiers : droits 500 (`dr-x-----`) — parcours et lecture, pas d'écriture.
- Fichiers du site : droits 400 (`-r-----`) — lecture seule.
- `/var/log/apache2/access.log` et `error.log` : droits 600 (`-rw-----`) — lecture et écriture pour les journaux.
- `/etc/apache2/` : réservé à `root:root`, droits 700 récursifs.

Cette configuration garantit qu'une éventuelle faille dans le site web ne permettrait pas à un attaquant d'écrire ou de modifier des fichiers système.

5.4.5 Configuration du VirtualHost

Le serveur Apache est configuré via le fichier `000-default.conf` dans `/etc/apache2/sites-available/`. Ce fichier définit le port d'écoute (80), le répertoire racine (`/var/www/html`), et les chemins des journaux d'accès et d'erreurs. Le fichier actif dans `sites-enabled/` est un lien symbolique vers ce fichier disponible.

5.5 Fonctionnement des échanges front-end / back-end

Les pages du site ne se rechargent pas lors des interactions de l'utilisateur. La communication entre le navigateur et le serveur s'effectue de manière asynchrone via l'API `fetch` de JavaScript :

- ❑ L'utilisateur interagit avec l'interface (bouton, formulaire).
- ❑ Le JavaScript effectue une requête HTTP vers un script PHP de l'API.
- ❑ Le script PHP traite la demande, interroge ou met à jour la base de données, puis retourne une réponse au format JSON.
- ❑ Le JavaScript interprète la réponse JSON et met à jour dynamiquement l'affichage sans rechargement de page.

Cette approche permet une interface fluide et réactive, adaptée à une utilisation depuis un smartphone, sans nécessiter de framework JavaScript lourd.

6. Configuration du service base de données (MariaDB)

6.1 Contexte et objectif

L'enregistrement fiable des données est une exigence centrale du projet : températures relevées toutes les 30 minutes, événements système, cycles de séchage, lots et variétés de houblon, masses produites. Une base de données relationnelle MariaDB (compatible MySQL, imposé par le cahier des charges) a été retenue pour structurer, stocker et restituer ces informations.

L'objectif est de permettre à l'agriculteur de disposer d'un historique complet et cohérent, utilisable à la fois pour le suivi en temps réel via l'interface web et pour l'analyse rétrospective des campagnes de séchage.

6.2 Périmètre de ma réalisation

Mon travail a couvert la conception du schéma relationnel, la création des tables avec leurs contraintes d'intégrité (clés primaires, clés étrangères, valeurs nullables), ainsi que la définition des stratégies d'enregistrement pour chaque table. J'ai également rédigé la documentation technique de la base.

6.3 Architecture de la base

La base de données, utilisée par le compte www-data, est organisée autour de neuf tables couvrant l'ensemble du cycle de vie d'un lot de houblon :

Table	Description
pause	Enregistre les pauses horodatées survenant durant le fonctionnement du séchoir.
variete	Référentiel des variétés de houblon disponibles et désactivées.
masse	Enregistre la masse de houblon produite à l'issue du séchage.
etage	Données statiques représentant les étages physiques du séchoir.
etatSechoir	Table d'état courant du séchoir (variable globale persistante).
capteur	Référence les capteurs de température actifs et inactifs du système.
lot	Table centrale — suit un cycle complet de production pour une variété donnée.
temperature	Stocke les mesures de température relevées par les capteurs.
lotEtage	Table d'association reliant un lot à son étage courant, traçant ses changements d'étage.

6.4 Modèle Conceptuel de Données (MCD)

Un Modèle Conceptuel de Données est structuré autour de trois éléments : les **entités**, les **associations** et les **attributs**. Une entité représente un objet du système (par exemple un lot de houblon), ses attributs décrivent ses propriétés (variété, taux de remplissage, durée de séchage), et les associations modélisent les liens entre entités.

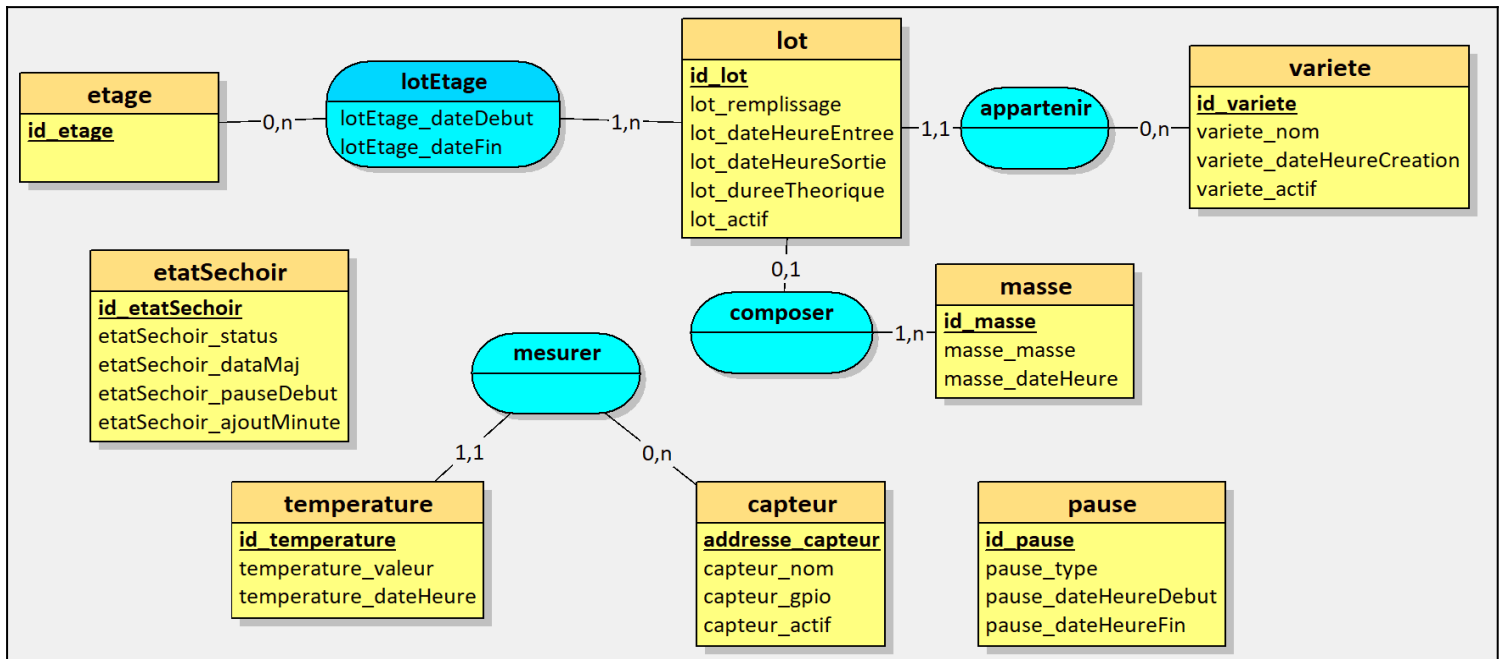
La lecture des associations se fait via les cardinalités, exprimées selon la notation suivante : (1,1) = 1 et 1 seul,

(0,1) = 0 ou 1,

(1,n) = entre 1 et n,

(0,n) = entre 0 et n.

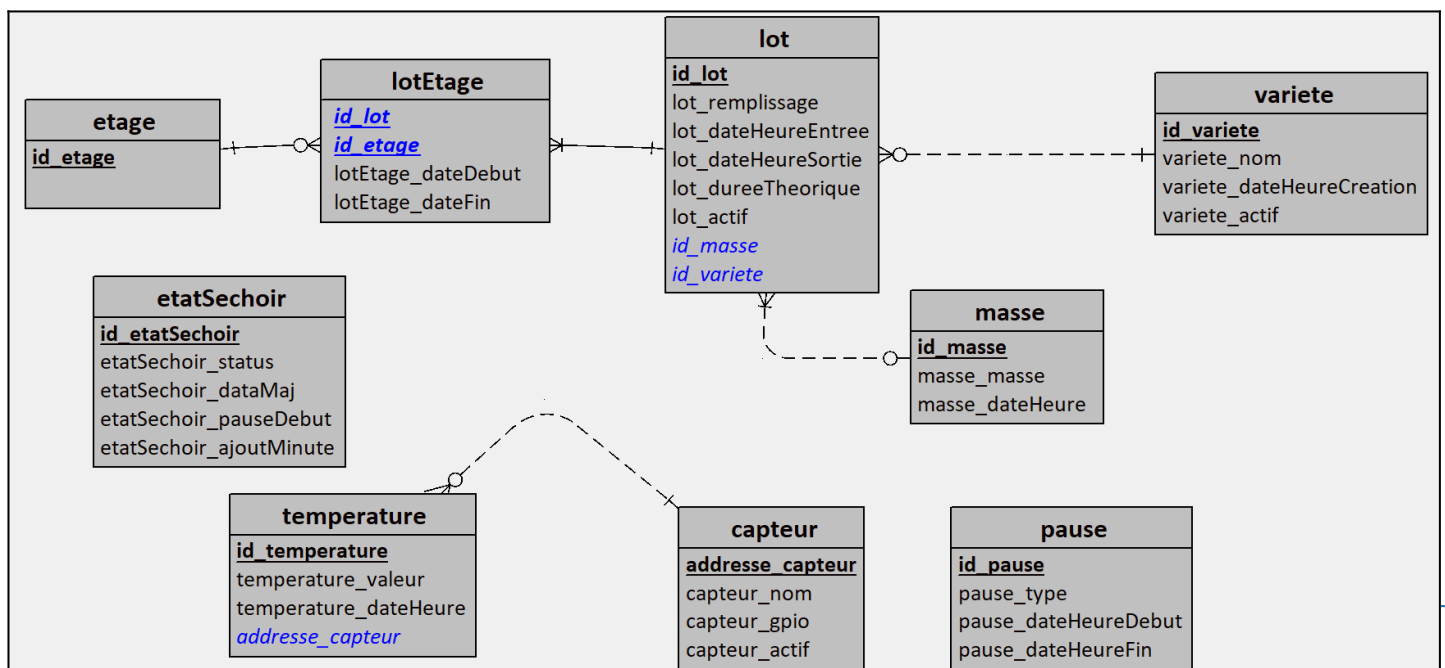
Par exemple, un lot appartient à une et une seule variété, tandis qu'une variété peut être associée à zéro à plusieurs lots.



6.5 Modèle Logique de Données (MLD)

Le Modèle Logique de Données est une traduction du MCD dans un format adapté à une implémentation en base de données relationnelle. Chaque entité devient une table, ses attributs deviennent des colonnes, et chaque table possède une clé primaire. Le passage du MCD au MLD dépend des cardinalités :

- (1,1) se traduit par une clé étrangère dans l'une des deux tables.
- (1,n) se traduit par une clé étrangère du côté « n ».
- (n,n) nécessite la création d'une table intermédiaire.



6.6 Ajout d'une variété

Pour permettre à l'agriculteur d'ajouter une nouvelle variété de houblon directement depuis l'interface, j'ai développé un script PHP effectuant une insertion en base via PDO. Avant l'insertion, une vérification est réalisée pour éviter les doublons. Si une variété portant le même nom existe déjà, l'insertion est annulée et un message d'erreur est retourné à l'interface.

La requête d'insertion est la suivante :

```
INSERT INTO variete (variete_nom, variete_active)
VALUES ('Cascade', 1)
```

Le champ `variete_active` est initialisé à 1 afin que la variété soit immédiatement disponible dans les listes déroulantes.

7. Configuration de l'horloge temps réel (RTC)

7.1 Contexte et objectif

Le séchoir est déployé dans une zone blanche, sans accès à Internet. La Raspberry Pi ne peut donc pas synchroniser son horloge via un serveur NTP. Or, l'horodatage correct de toutes les mesures et événements est une condition de fiabilité du système : des horodatages erronés rendraient l'historique inexploitable.

L'ajout d'une horloge temps réel (RTC) permet de conserver l'heure exacte même après une coupure d'alimentation, garantissant ainsi la cohérence des données à chaque redémarrage.

7.2 Solution retenue

Au cours du développement, une Raspberry Pi 5 a été mise à disposition. Contrairement à la Raspberry Pi 3 initialement prévue, la Pi 5 intègre nativement une RTC interne. Il suffit de brancher une pile sur le connecteur BAT de la carte pour alimenter cette RTC lorsque le système est hors tension, sans nécessiter de module externe ni de configuration I2C supplémentaire.

7.3 Périmètre de ma réalisation

J'ai réalisé une étude comparative des modules RTC externes courants (DS1307, DS3231, PCF8523) afin d'argumenter le choix technique initial. J'ai ensuite procédé à la configuration système : désactivation du faux RTC logiciel (fake-hwclock), réglage manuel de l'heure, écriture dans la RTC (hwclock -w), et vérification du bon fonctionnement après coupure d'alimentation.

7.4 Étude comparative des modules RTC externes

Avant la mise à disposition de la Raspberry Pi 5, une analyse des modules RTC disponibles a été conduite afin de sélectionner le composant le plus adapté : déploiement en zone blanche, environnement agricole soumis à des variations de température, et besoin de fiabilité sur une longue période sans maintenance.

Critère	DS1307	DS3231	PCF8523
Interface	I2C	I2C	I2C
Tension	5V	3,3V	3,3V
Précision	Moyenne	Haute (± 2 ppm)	Moyenne

Rétention pile	Variable	~6 ans	~6 ans
Coût moyen	~ 6,50 €	~ 7,90 €	~ 8,30 €
Compatible GPIO Raspberry Pi 3,3V	Non (adaptateur requis)	Oui	Oui

Conclusion : pour un usage critique en zone blanche où l'horodatage doit rester fiable sur toute une saison agricole, le DS3231 aurait été retenu. Sa tolérance aux variations thermiques et sa haute précision (± 2 ppm $\approx \pm 1$ minute par an) en font le module le plus adapté. La mise à disposition de la Raspberry Pi 5 a cependant rendu cette sélection caduque.

7.5 Analyse des risques liés à l'absence de RTC

Sans RTC fonctionnelle, à chaque coupure d'alimentation la Raspberry Pi redémarre avec une heure incorrecte. Les conséquences directes sont :

- Les mesures de température en base seraient horodatées incorrectement, rendant tout graphique d'évolution thermique inexploitable.
- Les événements en base (démarrage de cycle, alerte, arrêt) perdraient leur valeur chronologique, rendant impossible tout audit rétrospectif.

7.6 Procédure de configuration — Raspberry Pi 5 (RTC interne)

Étape 1 — Connexion de la pile

Brancher une pile bouton compatible sur le connecteur BAT de la Raspberry Pi 5. Cette pile alimentera exclusivement le circuit RTC lorsque la carte sera hors tension.

Étape 2 — Réglage initial de l'heure

Lors de la première mise en service, l'heure doit être renseignée manuellement puisque le système est hors réseau :

```
$ sudo timedatectl set-timezone Europe/Paris  
$ sudo timedatectl set-time "YYYY-MM-DD HH:MM:SS"
```

Étape 3 — Vérification du fonctionnement

Après un redémarrage, la commande suivante doit afficher l'heure correcte :

```
$ sudo timedatectl status
```

Étape 4 — Vérification finale après coupure

Pour valider le comportement lors d'une vraie coupure d'alimentation : couper l'alimentation, patienter quelques minutes, remettre sous tension, puis exécuter `date` et `sudo timedatectl status` pour confirmer que l'heure est restée correcte.

7.7 Bilan

La RTC constitue un prérequis technique indispensable au bon fonctionnement du système dans un contexte hors réseau. La Raspberry Pi 5, en intégrant nativement ce composant, simplifie considérablement le câblage et la configuration par rapport à la solution initialement envisagée avec un module I2C externe. La procédure de configuration a été testée et validée : après coupure d'alimentation et redémarrage, l'heure système est correctement restituée depuis la RTC interne.

8. Développement de l'IHM de configuration du houblon

8.1 Contexte et objectif

Avant de démarrer un cycle de séchage, l'agriculteur doit renseigner un ensemble de paramètres : la variété de houblon chargée à chaque étage, le taux de remplissage, la température cible (min/max) et la durée prévue du cycle pour le lot qu'il est entrain de renseigner. Cette saisie doit être possible depuis un smartphone, sans compétence informatique particulière. Ces paramètres sont ensuite sauvegardés en base de données dès le lancement du cycle.

8.2 Fonctionnalités développées

- Sélection de la variété de houblon par étage (liste déroulante alimentée depuis la table variete).
- Saisie du taux de remplissage de l'étage (boutons – et +, par pas de 10 %, entre 0 et 100 %).
- Saisie de la durée du cycle au format hh:mm avec boutons d'ajustement par pas de 10 minutes.
- Affichage récapitulatif de tous les paramètres.
- Enregistrement en base de données via un script PHP.
- Adaptation de l'interface selon le contexte (création ou modification d'un lot existant).

8.3 Périmètre de ma réalisation

J'ai conçu et développé la page HTML/CSS/JavaScript correspondante, ainsi que le script PHP côté serveur chargé de valider les données saisies et de les persister en base. L'interface a été pensée pour une utilisation mobile (responsive) via la librairie « bootstrap », avec un retour visuel immédiat à l'utilisateur après chaque action.

8.4 Architecture technique de l'IHM

8.4.1 Structure des fichiers

L'interface de configuration se présente sous la forme d'un overlay (fenêtre modale) qui s'ouvre au clic sur le bouton de chaque étage depuis la page principale de gestion du séchoir. Ce choix technique évite une navigation entre plusieurs pages et permet à l'agriculteur d'agir rapidement, sans perdre de vue l'état général du séchoir.

Le code est découpé en deux fichiers PHP distincts : gestion_sechoir.php qui affiche la vue principale des quatre étages, et gestion_sechoir_overlay.php qui contient la fenêtre modale de configuration.

8.4.2 Architecture JavaScript modulaire

Côté JavaScript, j'ai adopté une architecture modulaire en séparant les responsabilités en plusieurs fichiers distincts, chargés de manière différée (defer) :

- GestionAPI.js : gestion des appels vers l'API REST de la Raspberry Pi (get_status.php, création/suppression de lot).
- GestionUI.js : manipulation du DOM, affichage et masquage de l'overlay, mise à jour des champs.
- GestionUTILS.js : fonctions utilitaires (validation du format hh:mm, calcul du pourcentage de remplissage).
- GestionSTATE.js : gestion de l'état courant de l'overlay (étage sélectionné, lot existant ou nouveau).
- index.js : point d'entrée, initialisation et liaison des événements.

Ce découpage facilite la maintenance et la lecture du code.

8.5 Comportement de l'overlay selon le contexte

L'overlay adapte son comportement selon qu'un lot existe déjà sur l'étage concerné ou non. Lors de l'ouverture, l'état courant de l'étage est récupéré depuis l'API : si un lot est déjà enregistré, les champs sont pré-remplis avec ses données et le bouton principal affiche « Modifier le lot » ; sinon, le formulaire est vide et le bouton affiche « Créer un lot ».

<Overlay sans houblon>

The screenshot shows a web application interface with a sidebar on the left containing a list of levels (étages) with arrows pointing down. The main area displays an overlay window titled "Étage 4" with a close button (X) in the top right corner. The overlay has a subtitle "Configuration d'un lot de houblon". Below this, a red message states "Cette étage ne contient pas de lot". The form contains the following elements:

- A dropdown menu for "Variété de houblon" with the text "-- Sélectionner une variété --".
- A "Remplissage" section with a value of "50" and percentage symbols, flanked by minus and plus buttons.
- A "Temps de séchage voulue" section with a text input field containing "00:00" and minus/plus buttons.
- Two buttons at the bottom: "Créer un lot" and "Supprimer ce lot".
- A "Quitter" button at the very bottom of the overlay.

In the bottom right corner of the application, there is a status area showing "CAPTEUR 6", a temperature of "18.3°C", and the time "09.13".

<Overlay avec houblon>

The screenshot shows a web application interface with a modal window titled 'Étage 4' (Stage 4) for hop configuration. The modal has a close button (X) in the top right corner. Inside the modal, the title 'Étage 4' is underlined. Below it, the text 'Configuration d'un lot de houblon' (Configuration of a hop lot) is displayed. A green message states 'Cette étage contient un lot' (This stage contains a lot). The lot is identified as 'LOT_66'. The 'Variété de houblon' (Hop variety) is set to 'Cascade' in a dropdown menu. The 'Remplissage' (Filling) is set to '50 %' with minus and plus buttons. The 'Temps de séchage voulue' (Desired drying time) is set to '00:00' with minus and plus buttons. At the bottom of the modal, there are two buttons: 'Sauvegarder le lot' (Save the lot) and 'Supprimer ce lot' (Delete this lot). A 'Quitter' (Quit) button is located at the bottom of the modal. In the background, a sidebar on the left shows a tree view with 'veau h', 'veau m', 'veau b', and 'veau tir'. On the right, a temperature sensor 'CAPTEUR 6' shows '18.3°C' and '09.13'.

Les champs 'variétés', 'remplissage', 'temps de séchage' sont chargé par rapport a la configuration du LOT_66.

8.6 Communication avec l'API

La sauvegarde des paramètres s'effectue via une requête fetch vers un endpoint PHP côté serveur. Les données sont envoyées au format JSON et le script PHP les valide puis les insère ou les met à jour en base de données.

Script de connexion a la base de donnée

```
2 function db_connect(): ?PDO
3 {
4     $host = "localhost";
5     $dbname = "base_sechoir";
6     $user = "dbsechoir";
7     $pass = "password";
8     $pdo = null;
9
10    try
11    {
12        $pdo = new PDO(
13            "mysql:host=$host;dbname=$dbname;charset=utf8",
14            $user,
15            $pass,
16            [
17                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
18                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
19            ]
20        );
21
22        return ($pdo);
23    }
24    catch (PDOException $e)
25    {
26        die("Erreur connexion BDD : " . $e->getMessage());
27        return (NULL);
28    }
29 }
```

Cette fonction retourne un objet « PDO » qui est une interface de communication avec la base de donnée.

Le « ? » permet de dire que cet objet peut être NULL si une erreur est survenue lors de la tentative de connexion.

Appeler cette fonction permet de récupérer un moyen de communiquer avec la base de données.

Le bloc **try...catch** : Il sert à capturer les erreurs de connexion (exceptions). Cela est crucial pour la sécurité, car cela empêche PHP d'afficher par erreur vos identifiants (nom d'utilisateur et mot de passe) sur l'écran de l'utilisateur en cas de panne du serveur.

L'option **PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION** : Indispensable pour le développement, elle force PHP à générer une erreur fatale (exception) dès qu'une requête SQL est mal écrite, rendant le débogage beaucoup plus rapide.

L'option **PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC** : Cette configuration définit le mode de récupération par défaut. Elle permet de recevoir les données sous forme de tableaux associatifs (clé => valeur) automatiquement, sans avoir à le préciser à chaque requête.

L'encodage **charset=utf8** : Il garantit que les caractères spéciaux et les accents soient correctement lus et enregistrés dans la base de données dès l'établissement de la connexion.

Pour finir, l'instanciation **new PDO(...)** : C'est l'étape précise où PHP tente de se connecter physiquement à la base de données.

Script d'envoi de requête a une base de donnée

```
31 function db_query(PDO $pdo, string $sql, array $params = []): PDOStatement
32 {
33     $stmt = $pdo->prepare($sql);
34     $stmt->execute($params);
35     return ($stmt);
36 }
37 ?>
38 |
```

Cette fonction permet d'envoyer une requête a une base de donnée.

Elle retourne un objet « PDOStatement » qui est le resultat de la requete executer et/ou contient les erreur survenue lors de la requête.

Le paramètre \$params = [] (Argument optionnel) : C'est une sécurité très importante. Le fait d'utiliser un tableau vide par défaut permet deux choses :

- **Si tu ne mets rien :** La fonction comprend qu'il n'y a pas de variables à filtrer (ex: "Afficher tout").
- **Si tu mets des valeurs :** Elle les utilise pour remplir les "trous" de ta requête SQL en les nettoyant, ce qui protège ton site contre les **injections SQL** (piratage).

\$pdo->prepare(\$query_sql) (La préparation) : Avant d'envoyer les données, la fonction envoie le "plan" de la requête au serveur. La base de données analyse la structure de la commande sans l'exécuter, ce qui améliore les performances si on répète la même action.

\$stmt->execute(\$params) (L'exécution) : PHP envoie les données réelles pour compléter la requête. Si une erreur de syntaxe SQL est présente, c'est à ce moment-là que l'exception (l'erreur fatale) sera déclenchée grâce aux réglages faits dans db_connect.

8.7 Mise à jour automatique de l'affichage

La page de gestion du séchoir interroge régulièrement l'API get_status.php pour récupérer l'état en temps réel de chaque étage et des six capteurs de température. La fonction rafraichirStatus() est appelée au chargement de la page et relancée périodiquement. Les données reçues alimentent directement les éléments du DOM via les fonctions updateEtatCycle(), updateEtage() et updateTemperature(). La température moyenne des six capteurs est calculée côté client et affichée dans un bloc dédié.

8.8 Difficultés rencontrées et solutions apportées

La principale difficulté a été la gestion de la cohérence entre l'état affiché et l'état réel de la base de données, notamment lors d'actions rapides successives (création puis suppression immédiate d'un lot). J'ai résolu ce point en systématisant le rechargement des données depuis l'API après chaque action, plutôt que de modifier l'affichage localement.

Un autre point d'attention a été la validation du format de saisie de la durée (hh:mm). La validation est effectuée en JavaScript avant l'envoi, avec un retour visuel immédiat sous le champ concerné, sans bloquer l'utilisateur.

Et la dernière difficulté était qu'on était 2 étudiants a devoir développées une meme page, ce qui a nécessité une bonne coordination d'équipe pour l'avancement du projet.

9. Développement de l'IHM d'export de données

9.1 Contexte et objectif

À l'issue d'une saison de séchage, l'agriculteur doit pouvoir accéder à l'historique complet de ses productions : variétés séchées, dates et durées des cycles, températures relevées, masses finales obtenues. Le cahier des charges prévoit la visualisation de ces données sous forme de tableaux et graphiques, ainsi que leur export au format CSV.

9.2 Fonctionnalités développées

- Affichage des données de production historiques (variété, dates, masse en kg) sous forme de tableau.
- Visualisation graphique de l'évolution des températures par cycle de séchage.
- Filtrage des données par variété ou par numéro de lot.
- Export des données sélectionnées au format CSV.

9.3 Périmètre de ma réalisation

J'ai développé la page d'export dans son intégralité : interface HTML/CSS/JavaScript pour l'affichage et le filtrage, scripts PHP pour l'interrogation de la base de données et la génération du fichier CSV.

9.4 Architecture technique de la page

9.4.1 Parcours utilisateur en trois étapes

La page d'export est structurée selon un parcours utilisateur en trois étapes successives, matérialisées par trois sections qui s'affichent progressivement : le choix du type d'export, puis les filtres, puis les résultats. Cette progression guidée évite de surcharger l'écran et reste lisible sur smartphone.

Deux modes d'export ont été prévus : l'export par lot, qui permet d'obtenir le détail complet d'un cycle de séchage précis par lot (pleinement fonctionnel), et l'export par production, destiné à regrouper tous les lots d'une saison (en développement).

9.4.2 Chargement dynamique des données de filtrage

Dès le chargement de la page, la liste des variétés de houblon est récupérée automatiquement depuis la base de données via un appel à l'API `query_get_variete.php`. Les options du filtre sont générées dynamiquement par rapport aux type d'export.

9.4.3 Filtrage et prévisualisation des résultats

Lorsque l'utilisateur clique sur « Rechercher », la fonction `loadTable()` collecte les valeurs des champs de filtre actifs et les envoie au script PHP `query_get_lots_preview.php` via une requête POST. Les filtres sont collectés par une fonction générique `getFiltersRender()` qui parcourt un dictionnaire de correspondances entre les noms de champs logiques et les identifiants HTML, rendant le code extensible sans modification de la logique de collecte.

Les résultats sont affichés sous deux formes simultanées générées par la même fonction `renderTable()` : un tableau classique pour les écrans larges, et un affichage en cartes empilées pour mobile. Les statuts des lots (Terminé, En cours, Erreur) sont mis en valeur par des badges colorés définis dans le dictionnaire `BADGE`.

9.4.4 Génération et téléchargement du fichier CSV

L'export d'un lot individuel est déclenché par le bouton CSV de chaque ligne du tableau. La fonction `exportRow()` envoie les filtres actifs ainsi que le numéro du lot ciblé à l'API `query_export_csv.php`, qui retourne un objet JSON structuré contenant quatre blocs de données : les informations du lot, le passage par étage (avec dates de début et de fin de séjour), les relevés de température, et les événements (pauses, arrêts).

La construction du fichier CSV est réalisée entièrement côté JavaScript, sans rechargement de page. Chaque bloc est précédé d'un en-tête de section (=== TEMPERATURES ===, etc.) pour faciliter la lecture dans un tableur. Les blocs températures et événements sont optionnels selon les cases cochées dans les filtres. Le fichier générée est proposé au téléchargement.

	A	B	C	D	E	F	G
1	=== LOT ===						
2	id_lot	lot_remplissage	lot_dateHeureEntree	lot_dateHeureSortie	lot_dureeTheorique	lot_actif	variete_nom
3	39	80	2026-04-29 14:14:32	2026-04-29 14:14:39	30	0	Cascade
4							
5	=== ETAGES ===						
6	id_lot	id_etage	lotEtage_dateDebut	lotEtage_dateFin	duree_minute		
7	39	1	2026-04-29 14:14:38	2026-04-29 14:14:39	0		
8	39	2	2026-04-29 14:14:36	2026-04-29 14:14:38	0		
9	39	3	2026-04-29 14:14:32	2026-04-29 14:14:36	0		
10							
11	=== TEMPERATURES ===						
12	id_lot	id_temperature	temperature_valeur	temperature_dateHeure	adresse_capteur		
13	39	37	18.2	2026-04-29 14:14:32	CAP1		
14	39	38	18.3	2026-04-29 14:14:33	CAP1		
15	39	39	18.4	2026-04-29 14:14:34	CAP1		
16	39	40	18.6	2026-04-29 14:14:35	CAP1		
17	39	41	18.7	2026-04-29 14:14:36	CAP1		
18	39	42	18.8	2026-04-29 14:14:37	CAP1		
19	39	43	18.9	2026-04-29 14:14:38	CAP1		
20	39	44	19.0	2026-04-29 14:14:39	CAP1		
21							
22	=== EVENEMENTS ===						
23	id_pause	pause_type	pause_dateHeureDebut	pause_dateHeureFin			
24	1	pause_technique	2026-04-29 14:14:35	2026-04-29 14:14:36			
25	2	non	2026-04-29 14:14:35	2026-04-29 14:14:36			
26	3	non	2026-04-29 14:14:38	2026-04-29 14:14:40			
27	4	pause_technique	2026-04-29 14:14:38	2026-04-29 14:14:40			
28							
29							

9.5 Difficultés rencontrées et solutions apportées

La principale difficulté a été la structuration du fichier CSV de sortie pour qu'il soit à la fois exploitable dans un tableur et lisible en texte brut. L'ajout de séparateurs de sections textuels (=== LOT ===, === ETAGES ===, etc.) est un compromis qui facilite la compréhension sans perturber une importation automatisée, les en-têtes de colonnes étant systématiquement présents après chaque séparateur.

Une autre difficulté a été la gestion de l'état courant de l'interface entre la prévisualisation du tableau et le déclenchement de l'export. Deux fonctions distinctes ont été créées à cet effet : getFiltersRender() ignore les cases à cocher pour la prévisualisation, tandis que getFilters() les inclut pour la génération du CSV.

Conclusion

Ce dossier présente l'ensemble des réalisations dont j'ai eu la responsabilité dans le cadre du projet Séchoir à Houblon. Les cinq sujets traités — service web, base de données, RTC, IHM de configuration et IHM d'export — forment un ensemble cohérent couvrant l'intégralité de la chaîne : de la configuration de l'infrastructure système jusqu'à une partie de l'interface utilisateur finale.

Les principales difficultés rencontrées ont été la gestion de la cohérence des données entre l'interface et la base, la structuration d'un fichier CSV lisible par différents outils, et l'adaptation au changement de matériel (Raspberry Pi 3 vers Pi 5) en cours de projet. Ces défis ont été surmontés grâce à une approche incrémentale et à une documentation rigoureuse de chaque étape.

Présentation du travail du candidat N°2 – Ayoub Rabhi

Prototypage.

Mon prototypage ne repose pas sur une interface graphique (IHM utilisateur), mais sur une série de prototypes fonctionnels validés en console. Cette démarche itérative a permis de vérifier la précision des mesures, la fiabilité logicielle, puis la capacité matérielle à gérer une montée en charge sur deux bus 1-Wire.

Phase 1 : Validation de la mesure et de la logique d'alerte

- **Test 1 (Étalonnage matériel)** : Réalisation d'une première lecture brute et comparaison immédiate avec un thermomètre physique témoin. Ce prototype a validé la précision des sondes DS18B20.
- **Test 2 (Fiabilisation logicielle)** : Création d'un algorithme d'échantillonnage effectuant 5 mesures successives pour calculer une moyenne, permettant d'éliminer les erreurs de lecture ponctuelles.

```
/bin/bash 80x26
=== DEMARRAGE DU SYSTEME DE SURVEILLANCE ===
Cible : /sys/bus/w1/devices/28-00000689cb7e/w1_slave
-----
[Sonde Test Labo] Température moyenne : 26.74°C
[Sonde Test Labo] Température moyenne : 26.69°C
[Sonde Test Labo] Température moyenne : 26.62°C
[Sonde Test Labo] Température moyenne : 26.56°C
[Sonde Test Labo] Température moyenne : 26.90°C
[Sonde Test Labo] Température moyenne : 26.86°C
[Sonde Test Labo] Température moyenne : 26.69°C
[Sonde Test Labo] Température moyenne : 26.52°C
[Sonde Test Labo] Température moyenne : 26.34°C
[Sonde Test Labo] Température moyenne : 26.16°C
[Sonde Test Labo] Température moyenne : 26.05°C
[Sonde Test Labo] Température moyenne : 25.95°C
[Sonde Test Labo] Température moyenne : 25.85°C
[Sonde Test Labo] Température moyenne : 25.77°C
[Sonde Test Labo] Température moyenne : 25.66°C
[Sonde Test Labo] Température moyenne : 25.59°C
[Sonde Test Labo] Température moyenne : 25.49°C
[Sonde Test Labo] Température moyenne : 25.42°C
[Sonde Test Labo] Température moyenne : 25.35°C
[Sonde Test Labo] Température moyenne : 25.29°C
[Sonde Test Labo] Température moyenne : 25.24°C
[Sonde Test Labo] Température moyenne : 25.15°C
```

Cette interface valide l'algorithme de lissage. Bien que 5 mesures soient prises en interne, l'affichage est simplifié pour l'utilisateur.

- **Test 3 (Gestion des seuils) :** Ajout d'un module de surveillance comparant la température moyenne à des seuils Minimum et Maximum pour déclencher des alertes console.

```
/bin/bash 80x22
=== SYSTEME DE SURVEILLANCE AVEC ALERTES ===
Seuils configurés : 23.00C - 28.00C
-----
[Sonde Test] ALERT : Température trop HAUTE ! 30.850C
[Sonde Test] ALERT : Température trop HAUTE ! 28.110C
[Sonde Test] ALERT : Température trop BASSE ! 21.810C
[Sonde Test] ALERT : Température trop HAUTE ! 30.450C
[Sonde Test] ALERT : Température trop HAUTE ! 28.040C
[Sonde Test] OK : Température normale 26.470C
[Sonde Test] OK : Température normale 25.400C
[Sonde Test] OK : Température normale 24.380C
[Sonde Test] OK : Température normale 23.710C
[Sonde Test] OK : Température normale 23.250C
[Sonde Test] OK : Température normale 27.240C
[Sonde Test] OK : Température normale 25.880C
[Sonde Test] ALERT : Température trop BASSE ! 22.060C
[Sonde Test] ALERT : Température trop BASSE ! 21.940C
[Sonde Test] ALERT : Température trop BASSE ! 22.050C
[Sonde Test] ALERT : Température trop HAUTE ! 31.440C
[Sonde Test] ALERT : Température trop HAUTE ! 28.790C
[Sonde Test] OK : Température normale 27.450C
```

Prototype de l'interface d'alerte. Le logiciel détecte un dépassement de seuil et informe l'opérateur immédiatement via la console de monitoring.

Phase 2 : Prototypage de l'architecture Multi-Bus

L'enjeu de cette phase était de valider la stabilité du système lors de l'ajout progressif de capteurs.

- **Test 4 (Multiplexage) :** Test de 2 capteurs sur un même bus GPIO pour valider l'adressage unique.

```

/bin/bash 86x26
=== SURVEILLANCE DIFFÉRENCIÉE ACTIVÉE ===
-----
[Capteur 1 ] !! ALERTE FROID - ACTIVATION DU BRÔLEUR !! : 24.34°C (Seuil min: 25.0)
[Capteur 2 ] Température : 22.65°C
-----
[Capteur 1 ] !! ALERTE FROID - ACTIVATION DU BRÔLEUR !! : 24.38°C (Seuil min: 25.0)
[Capteur 2 ] Température : 22.69°C
-----
[Capteur 1 ] !! ALERTE FROID - ACTIVATION DU BRÔLEUR !! : 24.25°C (Seuil min: 25.0)
[Capteur 2 ] Température : 22.70°C
-----
[Capteur 1 ] !! ALERTE FROID - ACTIVATION DU BRÔLEUR !! : 24.22°C (Seuil min: 25.0)
[Capteur 2 ] Température : 22.74°C
-----
[Capteur 1 ] !! ALERTE FROID - ACTIVATION DU BRÔLEUR !! : 24.19°C (Seuil min: 25.0)
[Capteur 2 ] Température : 22.75°C
-----
[Capteur 1 ] Température : 27.75°C
[Capteur 2 ] Température : 24.20°C
-----
[Capteur 1 ] Température : 29.00°C
[Capteur 2 ] Température : 24.89°C
-----
[Capteur 1 ] Température : 30.24°C
[Capteur 2 ] Température : 24.70°C
-----

```

Cette interface de test confirme la capacité du logiciel à gérer le multiplexage. Chaque sonde est identifiée par son adresse hexadécimale, permettant une surveillance ciblée.

- **Test 5 (Architecture Double Bus)** : Séparation physique sur 2 bus GPIO avec 1 capteur par bus pour tester l'indépendance des flux.
- **Test 6 (Densification)** : Passage à 4 capteurs (2 par bus) pour vérifier l'absence de chutes de tension sur la ligne.
- **Test 7 (Configuration Cible)** : Prototype final avec 6 capteurs (3 par bus) et intégration du client SQL. Ce test confirme la capacité du système à couvrir l'ensemble du séchoir.

```

/bin/bash 80x15
=== SURVEILLANCE DOUBLE BUS (6 SONDES) ===
-----
temperature capteur : 1 : 23.299601
Bonne température !
temperature capteur : 2 : 23.812000
Bonne température !
temperature capteur : 3 : 22.574600
ATTENTION TROP FROID
temperature capteur : 4 : 22.899799
ATTENTION TROP FROID
temperature capteur : 5 : 22.875000
ATTENTION TROP FROID
temperature capteur : 6 : 22.875000
ATTENTION TROP FROID

```

C'est le tableau de bord final. On y voit les 6 températures stabilisées et le message de succès de connexion à la base de données. C'est la preuve que toute la chaîne technique est opérationnelle.

Bilan et limites du prototypage

La stratégie de prototypage par paliers a permis de sécuriser l'intégralité de la chaîne d'acquisition. Sur le plan matériel, la stabilité électrique du système a été validée par une montée en charge progressive jusqu'à six sondes. L'adoption d'une architecture en « Double Bus » est un succès, car elle garantit aujourd'hui que le matériel ne sature pas, assurant ainsi des relevés constants sans perte de signal sur les broches GPIO de la Raspberry Pi. Côté logiciel, la fiabilité des mesures est désormais assurée par l'algorithme d'échantillonnage de cinq mesures successives, tandis que la logique d'alerte en console est pleinement opérationnelle pour signaler tout dépassement de seuil.

Concernant la partie réseau, le dernier prototype a permis de confirmer que le « pont » avec l'environnement SQL est fonctionnel. À ce stade, la partie « Acquisition et Liaison » est techniquement validée : le système est capable de lire l'intégralité des capteurs du séchoir et d'établir une connexion stable. Pour cette étape, les tests de requêtes SQL ont été réalisés sur une structure intermédiaire afin de valider la communication, sans impacter pour le moment la base de données finale du projet.

Toutefois, dans le cadre de ce rapport final, certaines fonctionnalités n'ont pas encore pu être totalement déployées. Si la liaison technique est établie, l'étape de l'envoi automatique et systématique des données (via des requêtes INSERT) vers l'espace de stockage définitif reste à finaliser. Le programme constitue aujourd'hui une base technique robuste et opérationnelle, mais l'archivage en temps réel des mesures dans la base de production constitue la limite actuelle du développement.

Conception préliminaire.

Liste des objets :

Objet / blocs	Rôle(s)	Type / Classe
Connecteur SQL	Gère la liaison réseau avec le serveur MariaDB de l'Étudiant 2. Assure l'authentification sécurisée.	MYSQL *conn
Sélecteur de Configuration	Interroge la table lot créée par l'Étudiant 2 pour importer les seuils de température dans mon programme.	Fonction query_get_seuil()
Enregistreur de Données	Transmet les mesures de mes 6 sondes vers la table mesure. C'est l'objet qui alimente l'historique de l'Étudiant 2.	Requête INSERT INTO
Paramètres de Surveillance	Reçoivent les valeurs provenant de l'interface de l'Étudiant 2 pour mettre à jour la logique de mon programme en temps réel.	Variables seuilMin / seuilMax
Ordonnanceur de Flux	Définit la fréquence d'écriture (60s) pour garantir la performance de la base de données gérée par mon coéquipier.	Constante INTERVALLE_MESURE_SEC

Diagrammes de cas d'utilisations:

Diagramme de cas d'utilisations – Au démarrage

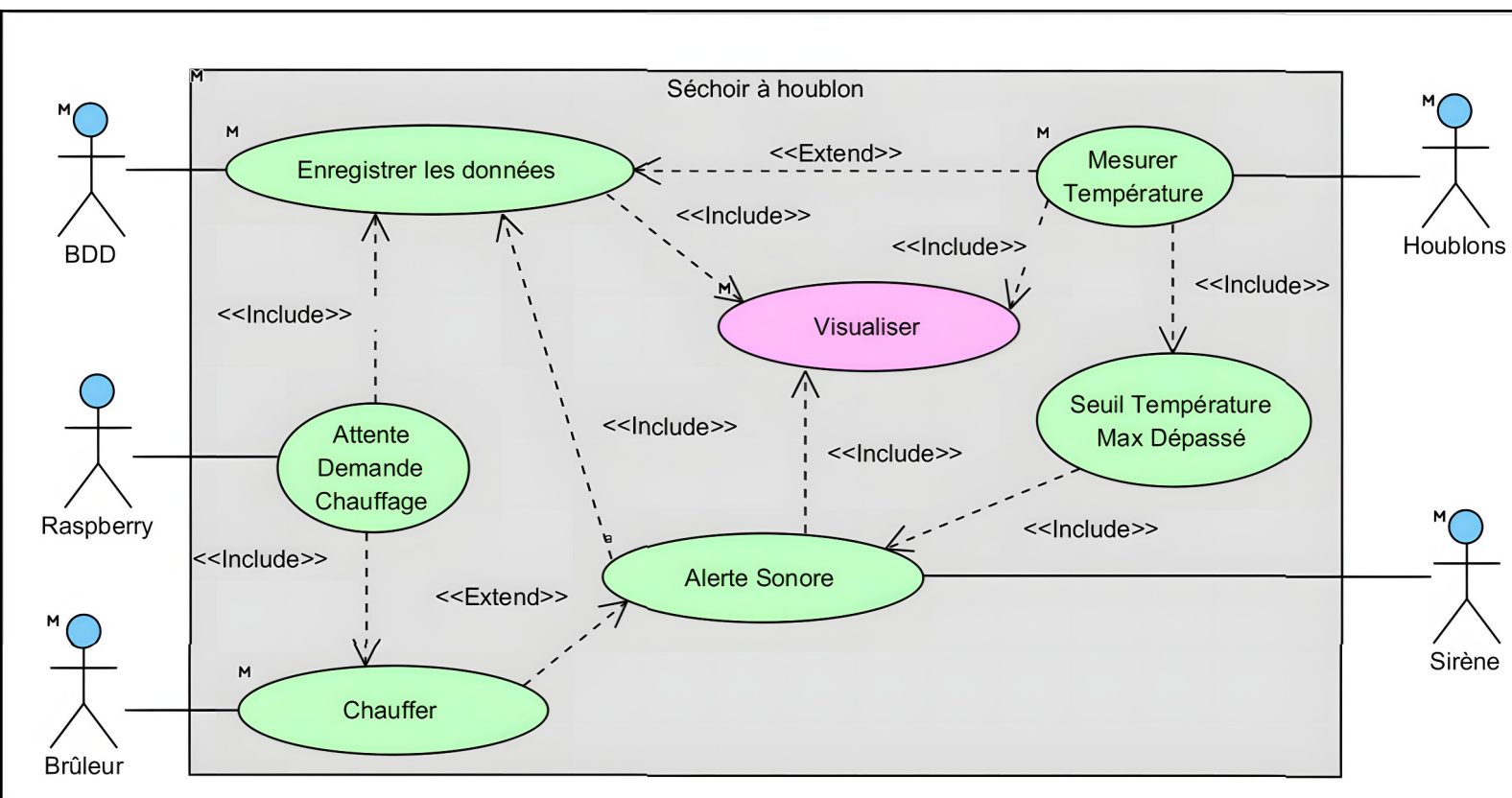
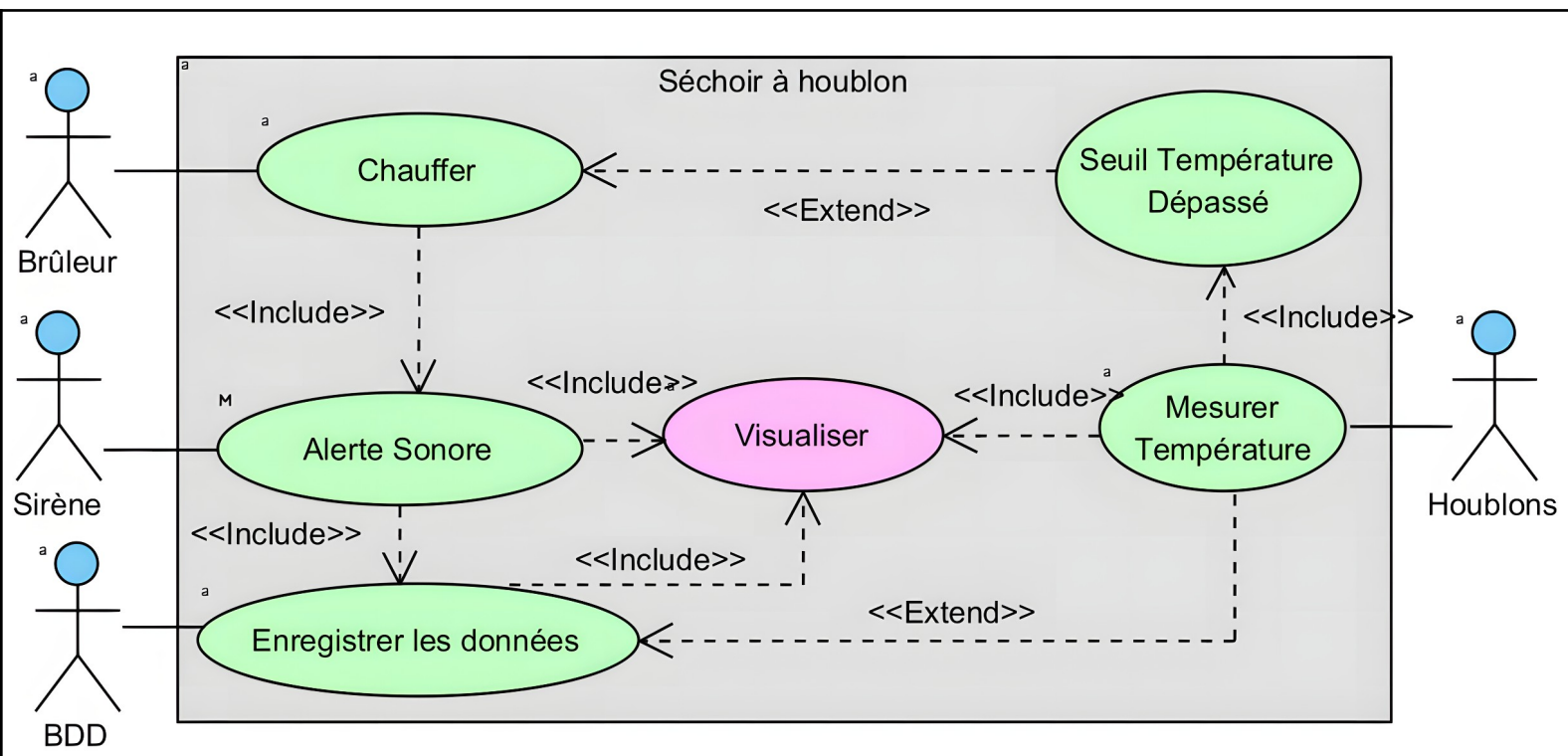
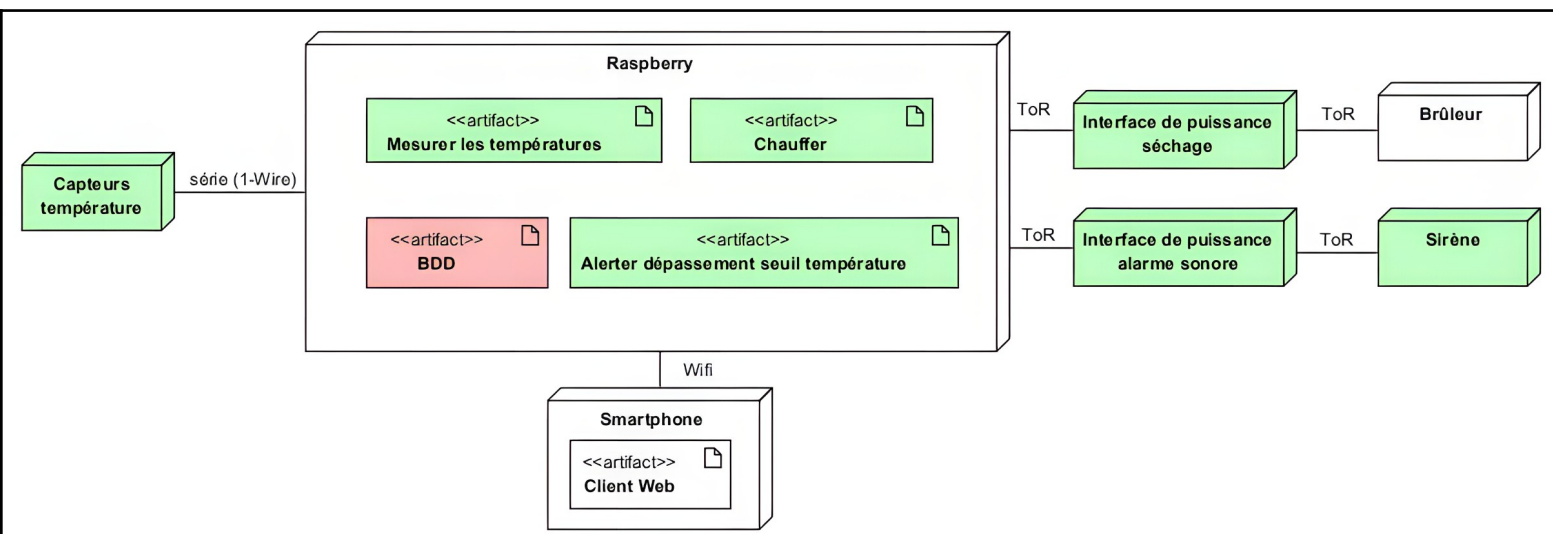


Diagramme de cas d'utilisations - Au démarrage



Diagrammes de déploiement :



Diagrammes d'exigences :

Diagramme d'exigences – Mesurer Température

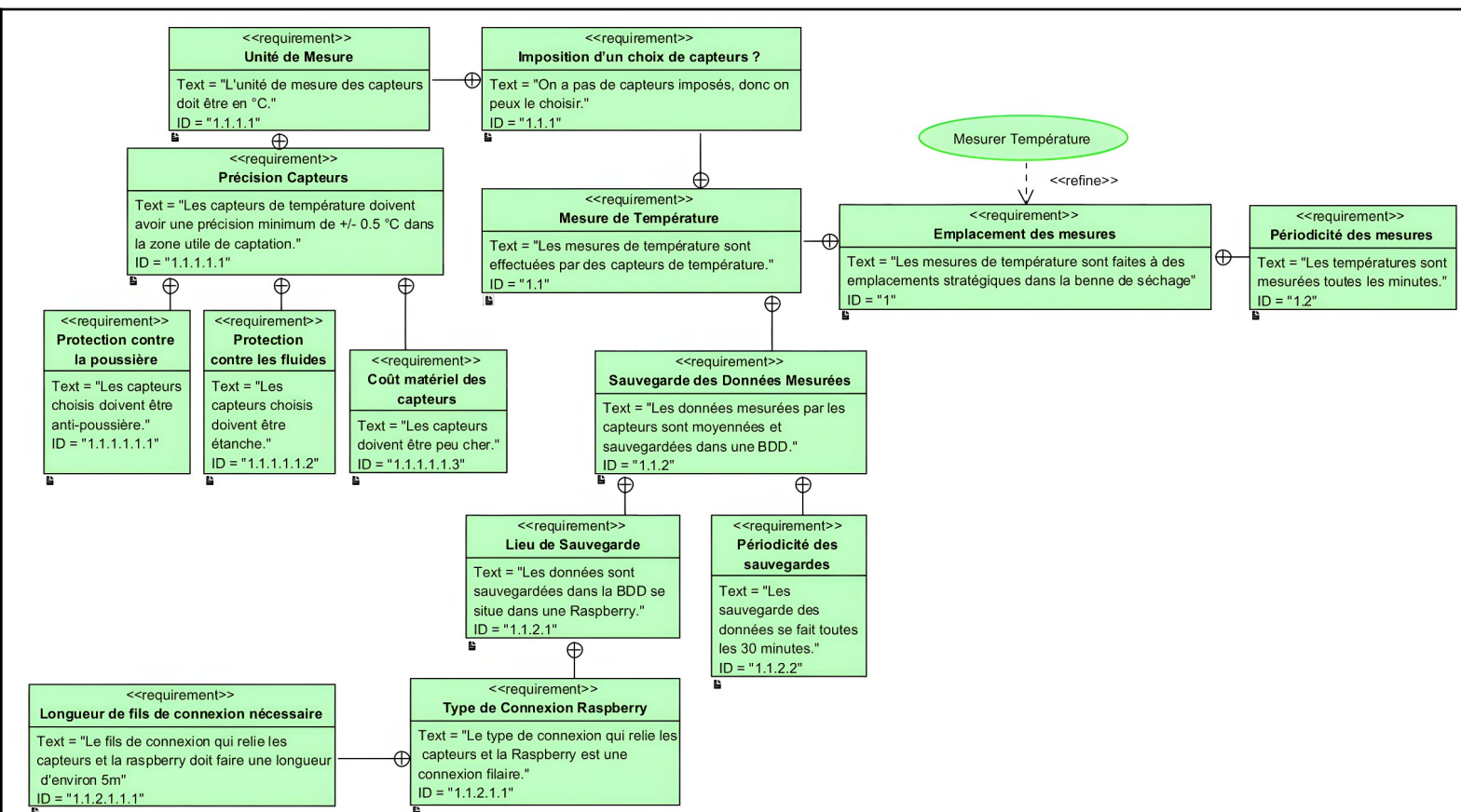


Diagramme d'exigences – Alerte Sonore

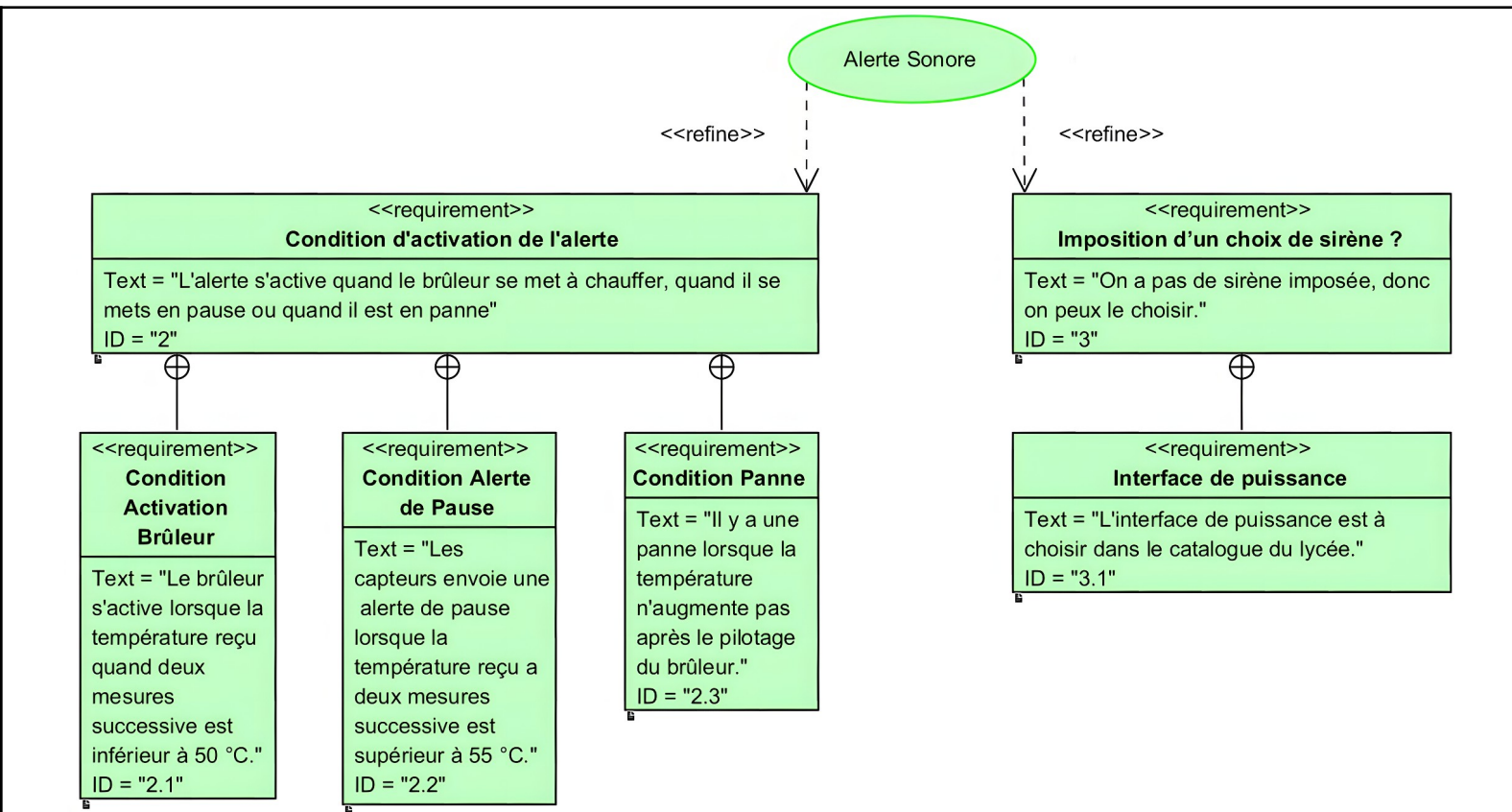


Diagramme d'exigences – Chauffer / Mettre en pause / Arrêter

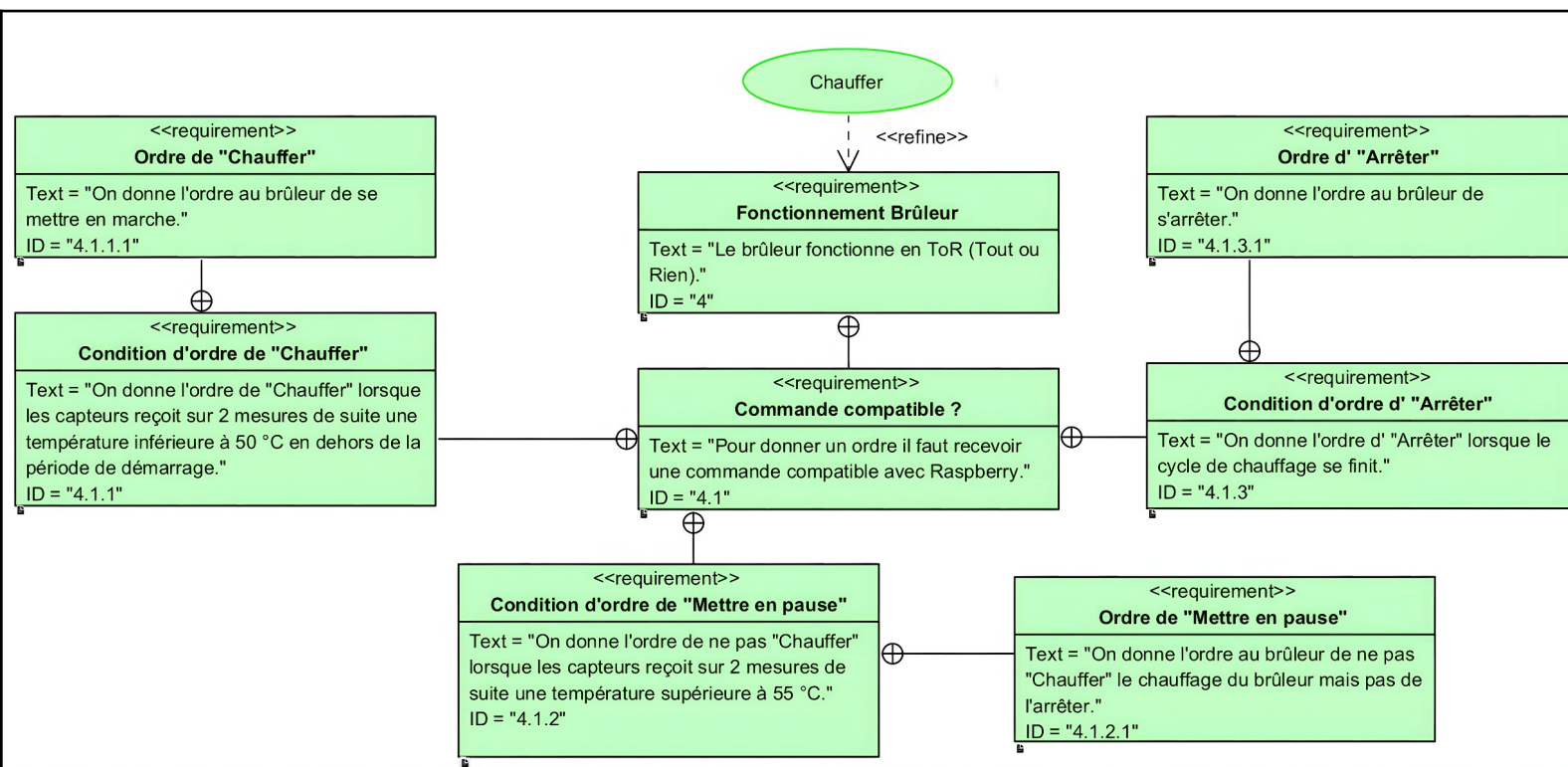


Diagramme d'exigences – Attente Demande Chauffage

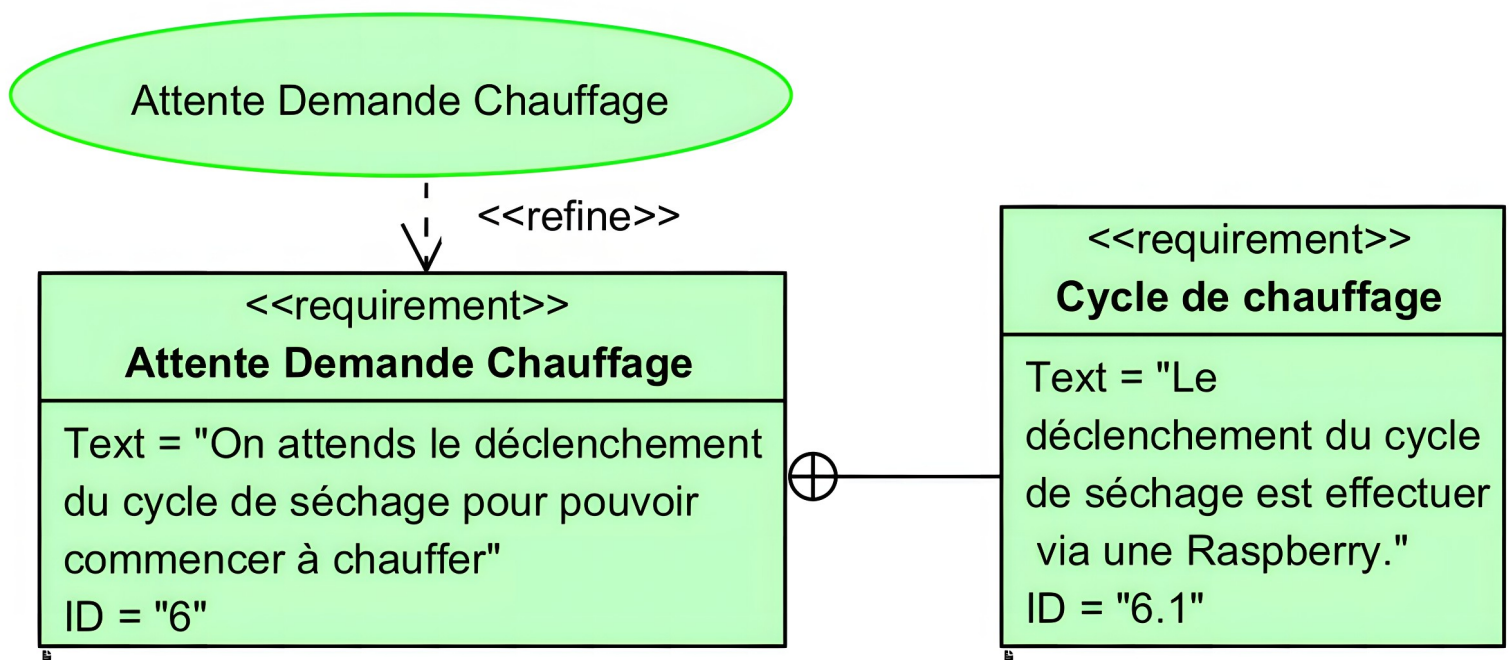
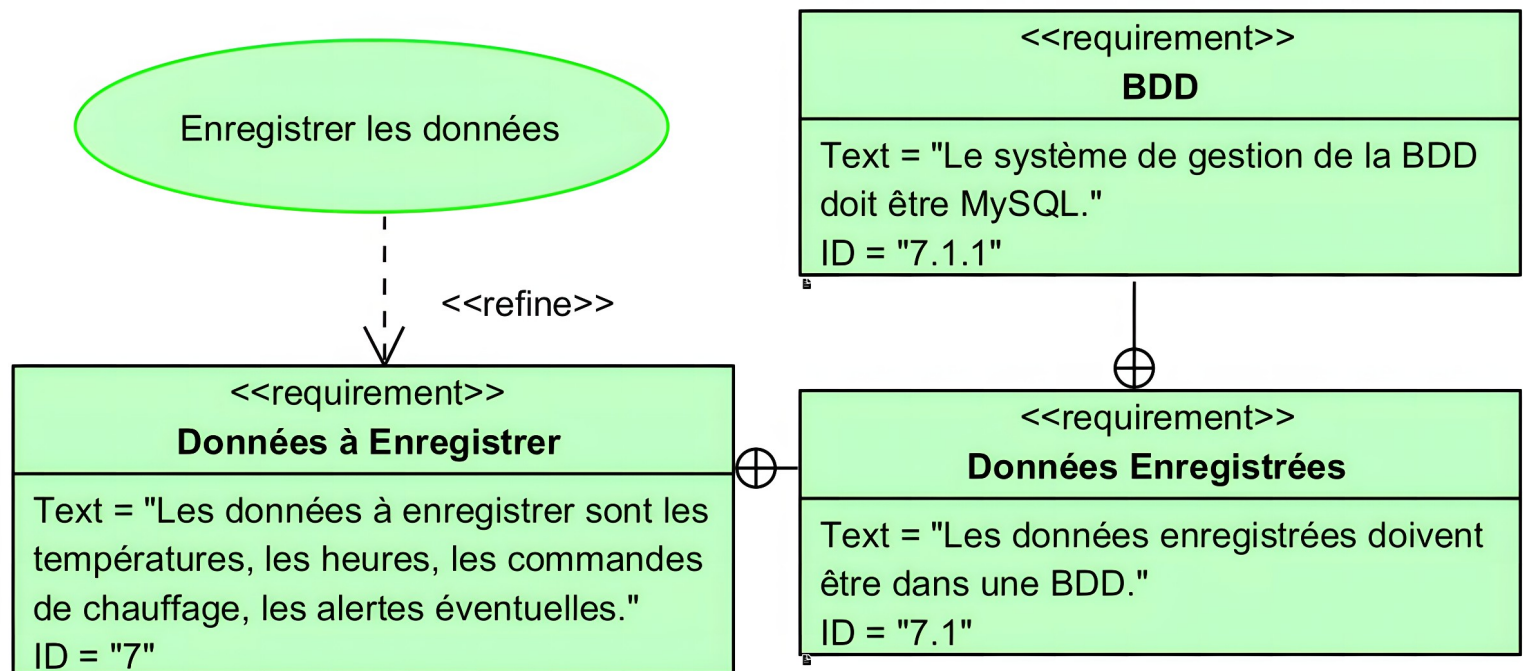


Diagramme d'exigences – Enregistrer les données



Diagrammes de séquences entre objet :

Diagramme de séquence – Au démarrage

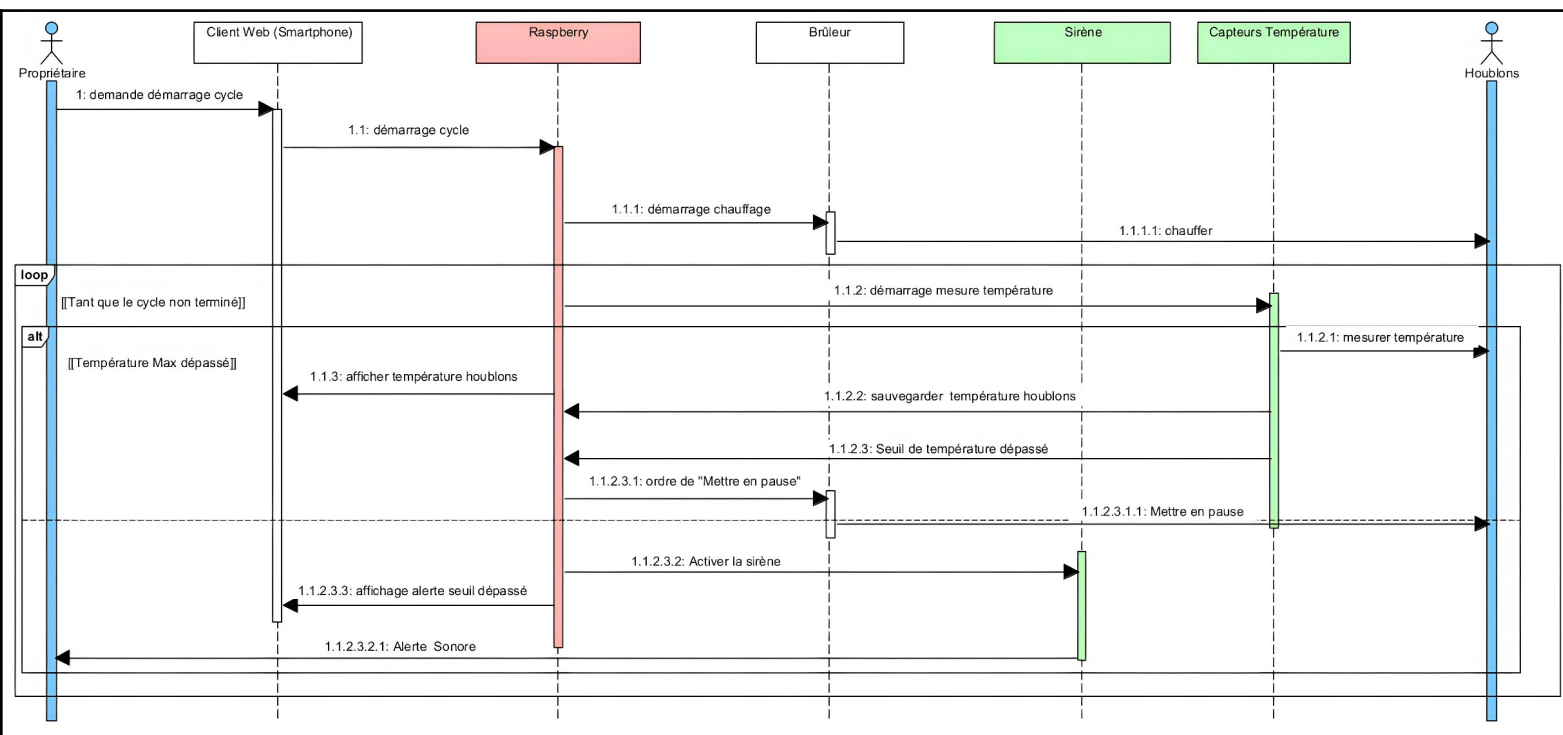


Diagramme de séquence – Durant le fonctionnement

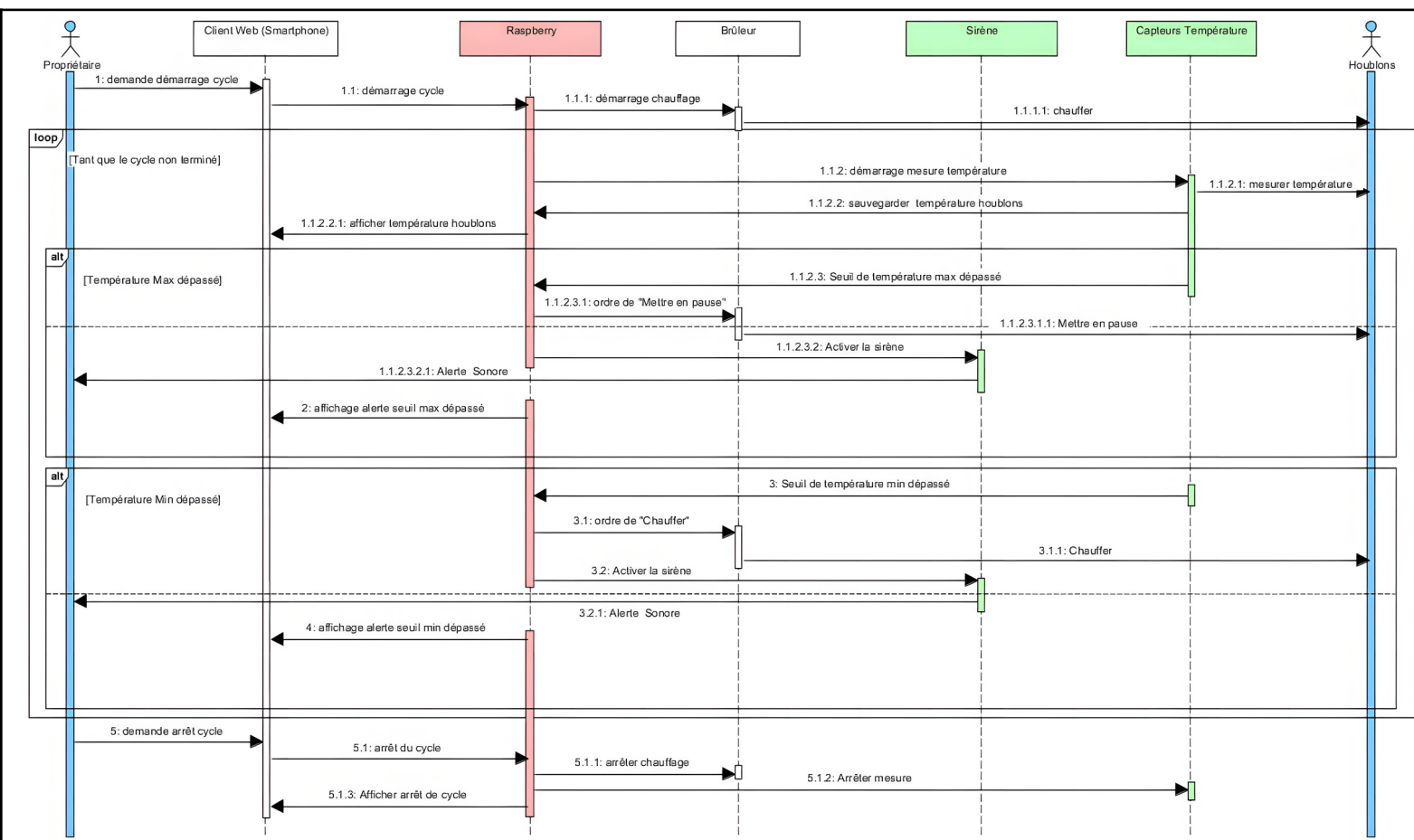
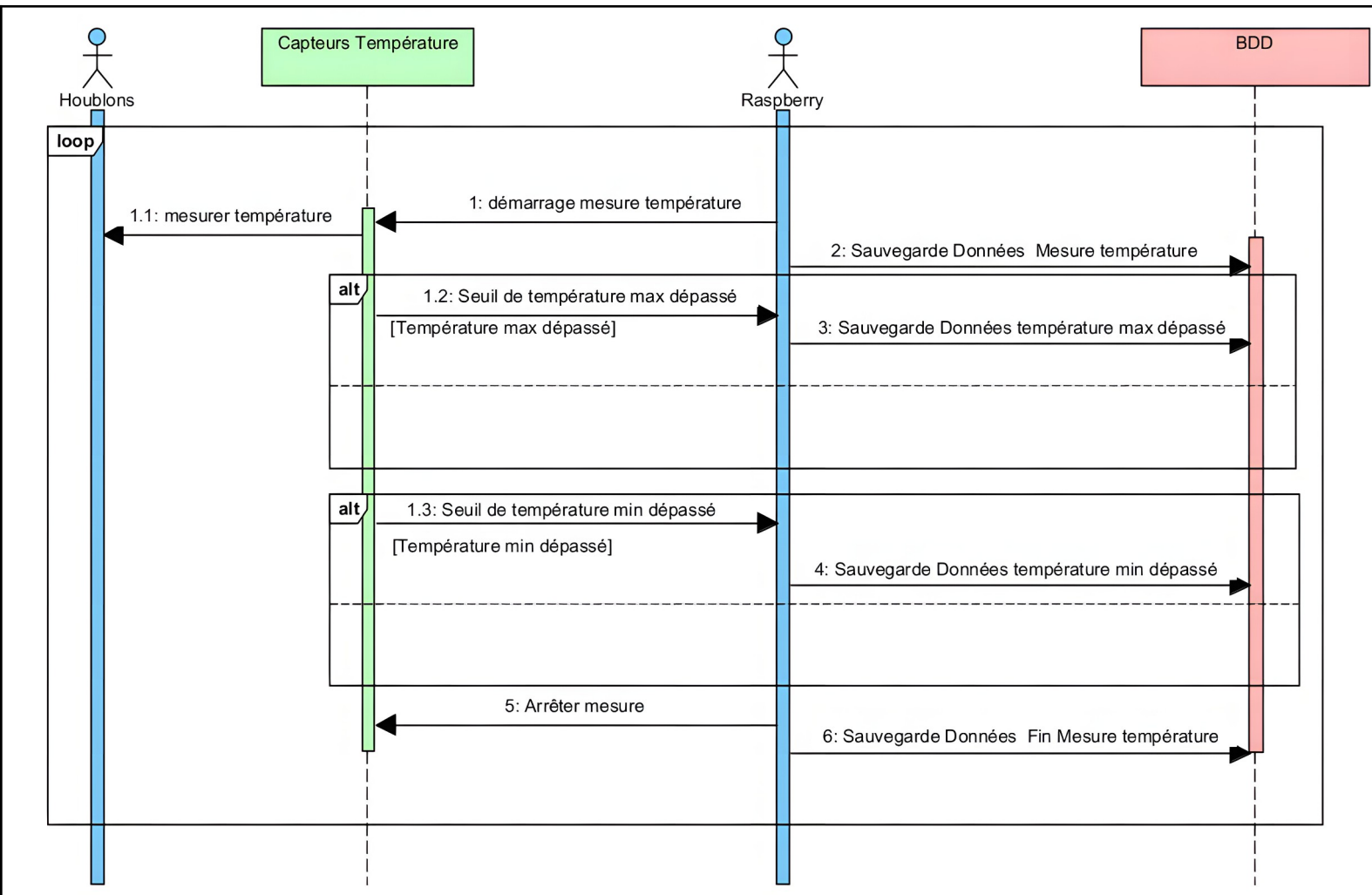


Diagramme de séquence – Mesurer Température / Sauvegarde Mesure

**Diagramme de séquence – Alerte Sonore**

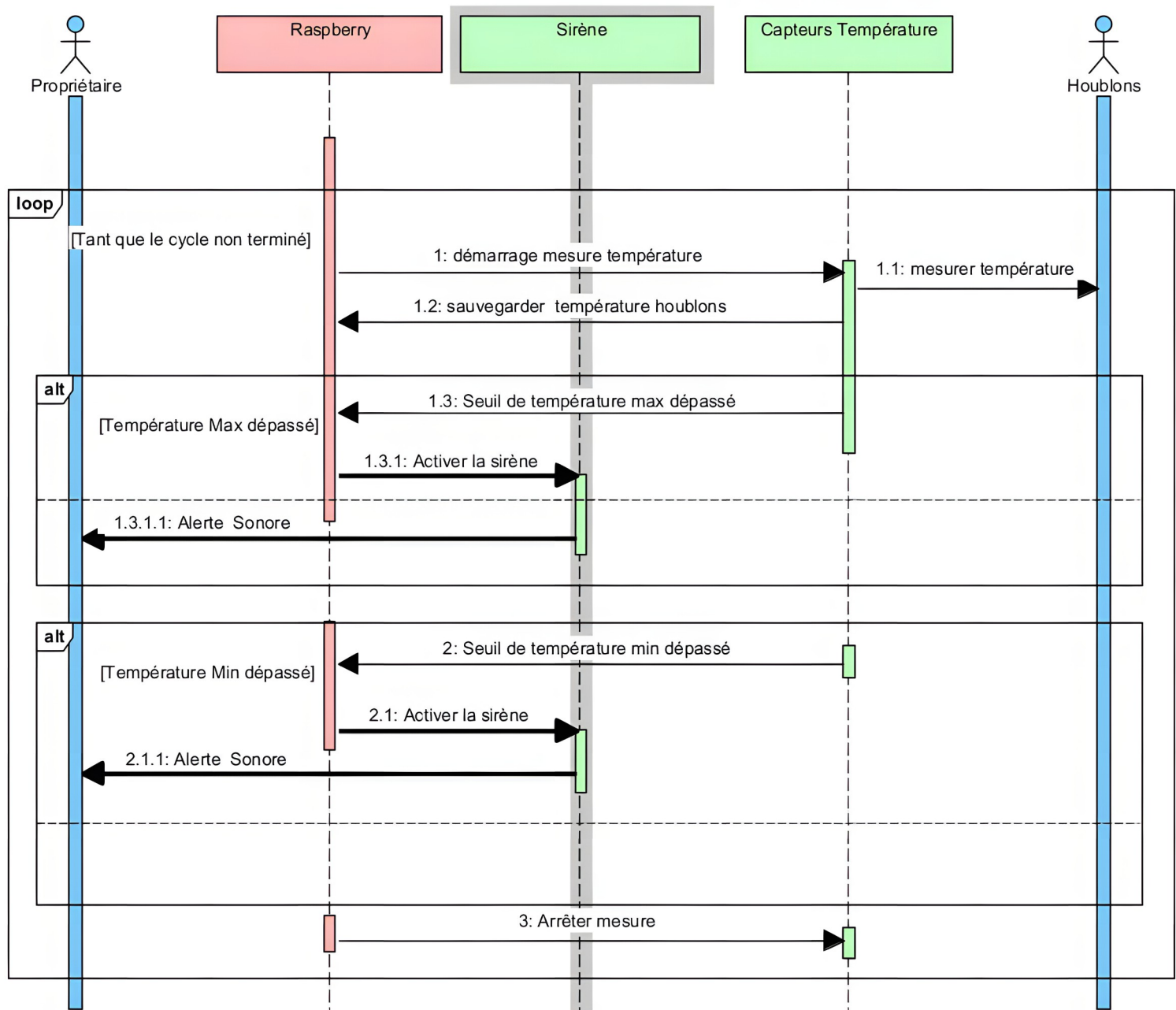


Diagramme de séquence - Chauffer / Mettre en pause / Arrêter

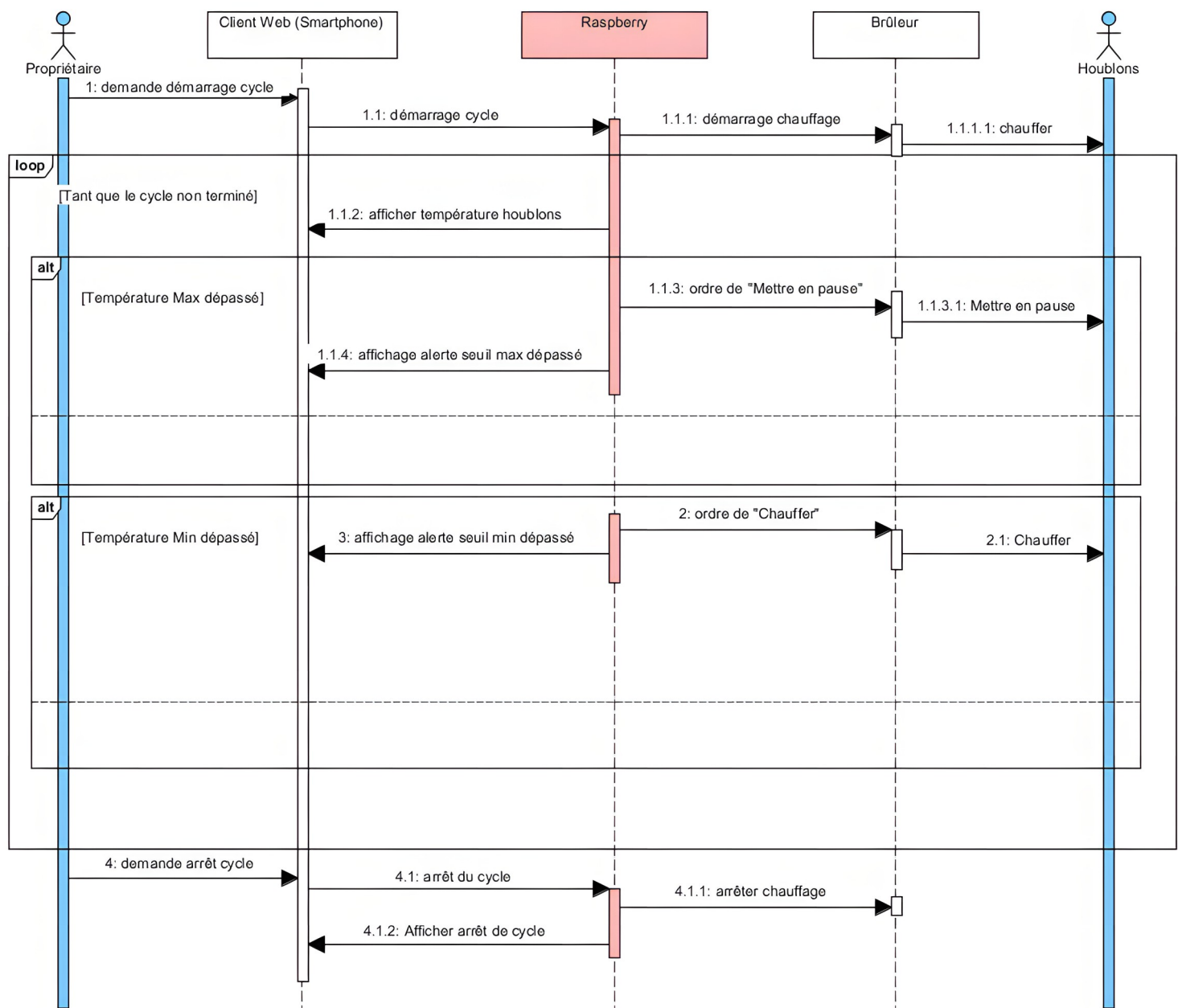
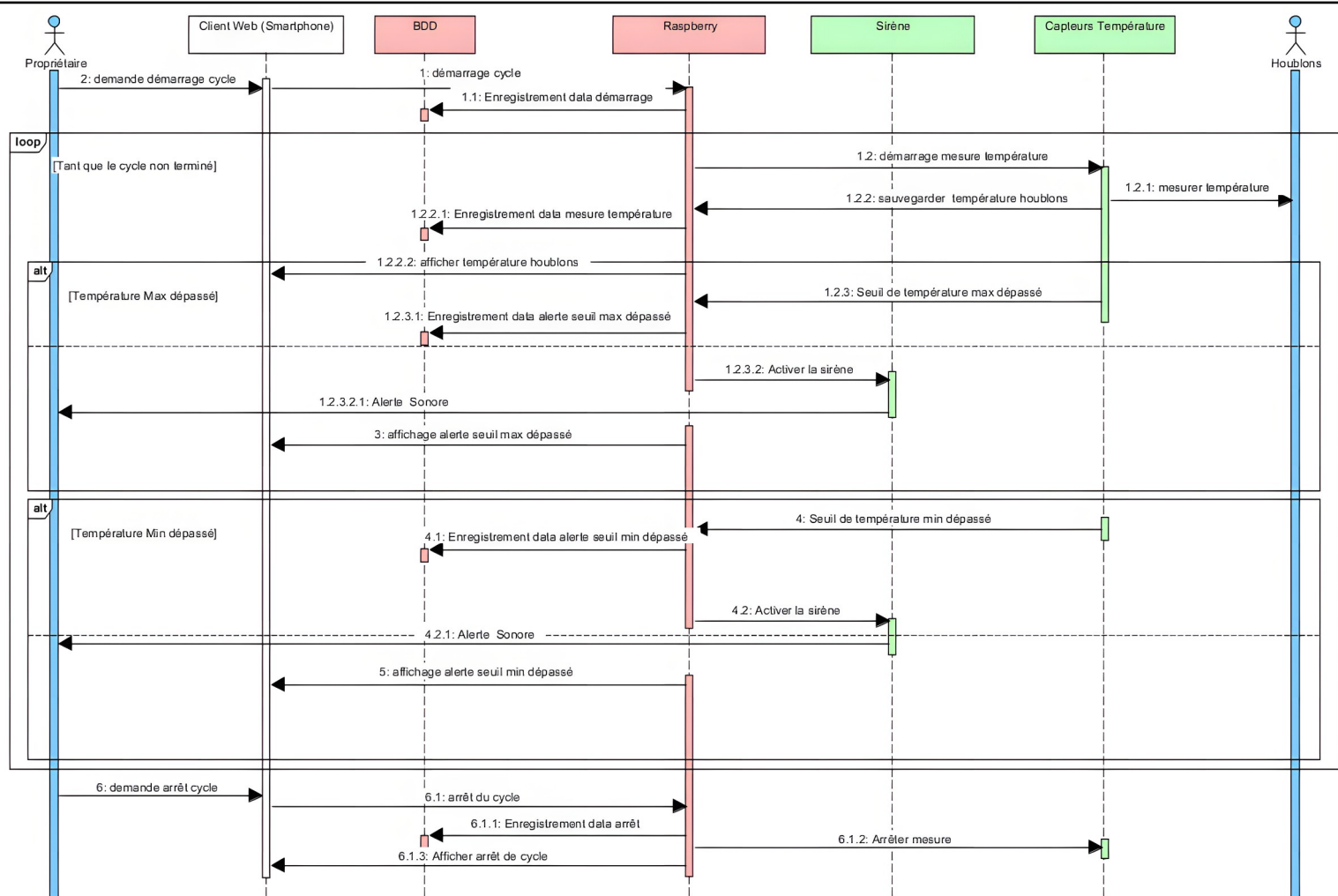


Diagramme de séquence – Enregistrer les données



Conception détaillée.

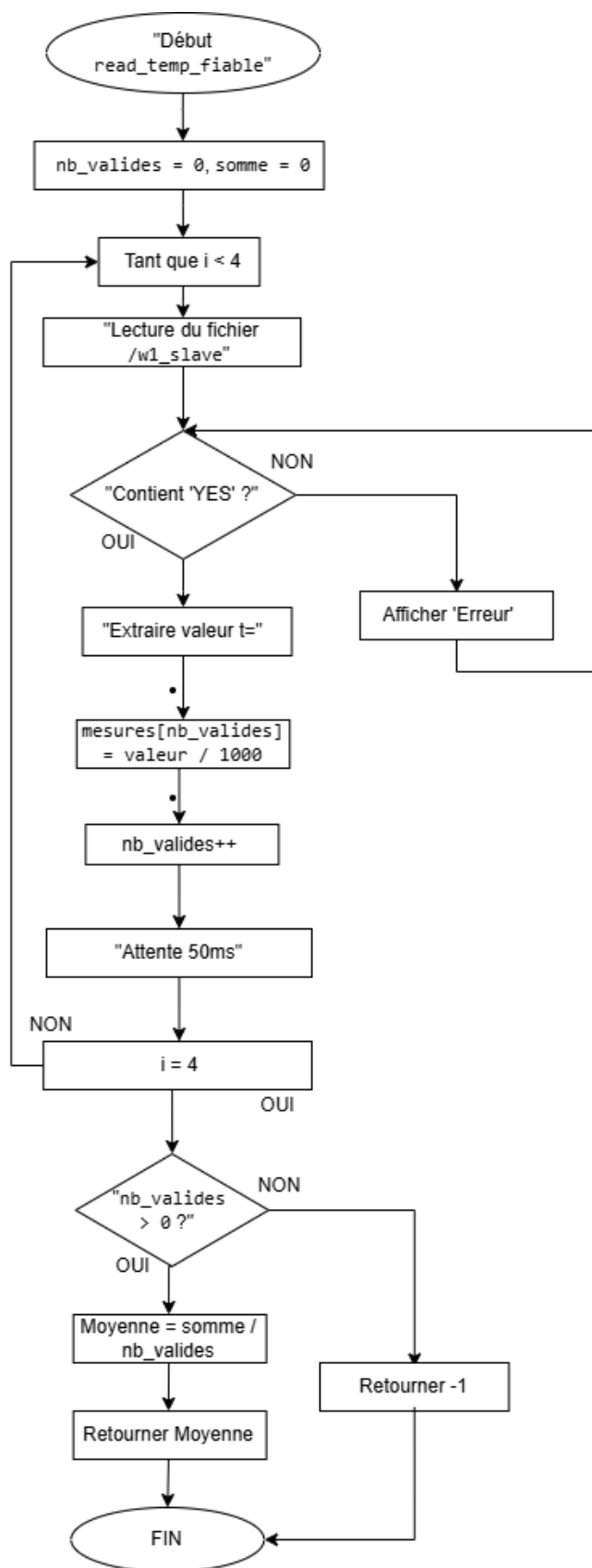
Étude du protocole et de la source de données

Le capteur **DS18B20** utilise le protocole **1-Wire**. Sous Linux (Raspberry Pi), chaque sonde génère un fichier virtuel `w1_slave`.

- ❑ **Analyse du composant** : L'étude du fichier montre que la donnée n'est valide que si la première ligne se termine par le code YES. La température brute se trouve après la chaîne `t=`.
- ❑ **Problématique** : Une lecture unique peut renvoyer une erreur ou une valeur aberrante en cas de parasite électrique sur les câbles.

Algorithme de la lecture fiabilisée (Méthode 'read_temp_fiable')

Pour garantir la précision demandée au cahier des charges, j'ai conçu un algorithme basé sur un cycle d'échantillonnage de 5 mesures.

Schéma explicatif du flux :

Pré-algorithme :**FONCTION** : lire_temperature_fiable**ENTRÉE** : chemin_capteur (Chaîne de caractères)**SORTIE** : temperature_moyenne (Nombre réel) ou Erreur**DÉBUT****INITIALISER** somme à 0**INITIALISER** nb_valides à 0**POUR** i allant de 1 à 5 **FAIRE** :**OUVRIR** le fichier chemin_capteur en mode lecture**LIRE** le contenu du fichier dans un buffer**FERMER** le fichier**SI** le buffer contient la chaîne "YES" **ALORS****EXTRAIRE** la valeur numérique après "t="**CONVERTIR** la valeur en Celsius (Division par 1000)**AJOUTER** la valeur à somme**INCRÉMENTER** nb_valides de 1**SINON****AFFICHER** "Erreur : Checksum invalide"**FIN SI** >> **ATTENDRE** 50 millisecondes**FIN POUR****SI** nb_valides est supérieur à 0 **ALORS****CALCULER** temperature_moyenne = somme / nb_valides**RETOURNER** temperature_moyenne**SINON****RETOURNER** -1 (Code erreur)**FIN SI FIN**

Plan de test unitaire de la lecture fiabilisée (Méthode 'read_temp_fiable') :

Action à tester	Tests à réaliser	Résultat(s) attendu(s)
Intégrité des données	Simuler un fichier sans la mention "YES".	Le programme affiche "Y a pas de YES" et ignore l'échantillon.
Précision du lissage	Effectuer une lecture stable (ex: 25°C).	Le programme retourne une moyenne précise au millième.
Robustesse matérielle	Saisie d'un mauvais chemin de capteur (NULL).	Le programme ne plante pas et passe à l'itération suivante.

Algorithme de surveillance et connectivité :

L'algorithme principal (main) gère la surveillance continue et prépare le lien avec le stockage SQL.

Algorithme de la méthode :

DEBUT

Initialiser les seuils par défaut (23°C / 28°C).

Tenter une connexion au serveur local MariaDB (root/password).

TANT QUE (Vrai) FAIRE :

POUR chaque capteur (de 1 à 6) :

Appeler read_temp_fiable().

Comparer la température reçue aux seuils (Min/Max).

Afficher l'état en console (Trop chaud / Trop froid / Bonne).

FIN POUR

Attendre l'intervalle de mesure.

FIN TANT QUE

FIN

Plan de test unitaire de surveillance et connectivité :

Action à tester	Tests à réaliser	Résultat(s) attendu(s)
Logique d'alerte	Forcer une température simulée à 30°C.	Affichage immédiat du message "ATTENTION TROP CHAUD".
Liaison SQL	Lancer le programme avec le service MySQL éteint.	Affichage d'une erreur via fprintf sans

		interrompre la boucle de lecture.
Adressage	Vérifier l'ID unique de la sonde 6.	Confirmation de l'ouverture du fichier 28-00000a54230c.

Étude de l'interface SQL

La conception utilise la structure MYSQL *conn. Bien que l'envoi définitif des données reste à finaliser, la méthode connexion_bdd a été conçue pour valider l'authentification et l'existence de la base base_sechoir. L'étude de la fonction query_get_seuil montre que le système est prêt à synchroniser ses alertes avec les paramètres enregistrés dans la table lot.

Codage.

Mon architecture repose sur le langage C (lecture bas niveau sur /sys/bus/w1/devices/) pour garantir une acquisition thermique précise et une réactivité immédiate de la sécurité machine. En complément, j'utilise des scripts Bash (automatisation système) pour assurer la fiabilité des sauvegardes et la pérennité de l'historique de production.

Les tests unitaires :

Action à tester	Tests à réaliser	Résultat(s) attendu(s)	Résultat(s) obtenu(s)	Remède(s)
Fiabilité de l'acquisition	Exécuter read_temp_fiab le (5 cycles de lecture par sonde).	Obtention d'une moyenne lissée éliminant les pics aberrants.	Conforme : Les micro-variations sont filtrées par le calcul de moyenne.	Aucun.
Intégrité du signal	Débrancher le fil de données (DATA) d'un capteur pendant le fonctionnement.	Détection de l'absence du code de contrôle "YES" dans le fichier système.	Conforme : Le programme affiche "Y a pas de YES" et ignore l'échantillon.	Gestion d'erreur logicielle par retour de valeur -1.
Architecture Double Bus	Répartir les 6 sondes sur deux lignes GPIO distinctes (3 par bus).	Lecture simultanée et stable sans chute de tension ni collision.	Conforme : La segmentation du bus a stabilisé le signal électrique.	Utilisation de deux répertoires /w1_slave distincts dans le code.
Liaison SQL	Initialiser la fonction connexion_bdd	Ouverture du socket et authentification	Conforme : Le "pont" avec la base	Vérifier que le service mysql est actif sur la

	vers le serveur MariaDB.	n réussie (root/password).	base_sechoir est établi.	Raspberry Pi.
Logique de surveillance	Simuler une température hors seuils (ex: 30°C pour un max de 28°C).	Déclenchement et affichage de l'alerte spécifique en console.	Conforme : Les messages "TROP CHAUD" ou "TROP FROID" sont instantanés.	Aucun.
Synchronisation des seuils	Appeler query_get_seuil pour mettre à jour les variables seuilMin/Max.	Récupération des valeurs configurées dans la table lot.	Conforme : Les seuils locaux sont mis à jour par les données de la BDD.	Aucun.
Archivage automatique	Automatiser l'envoi des relevés via une requête INSERT.	Stockage définitif des mesures dans la base pour l'interface web.	Non réalisé : La structure est prête mais l'envoi systématique n'est pas codé.	À implémenter dans la phase finale du développement.

Bilan personnel :

Planning réalisé :

*Le développement du logiciel de gestion (**test7.c**) et les tests du double bus ont été finalisés en fin de projet. Pour piloter mon travail, j'ai instauré une rigueur quotidienne en consignait chaque avancée dans mon **cahier de bord** (suivi journalier des tâches). Ce suivi m'a permis de maintenir une vision claire des objectifs atteints.*

Toutefois, mon planning présente un léger décalage par rapport aux prévisions initiales. Cet écart s'explique par des délais externes concernant la réception du **brûleur** (organe de chauffe) et des composants de la **sirène** (alerte sonore). Ce contretemps a nécessité une réorganisation des phases de tests matériels en fin de session.

Difficultés rencontrées :

La réalisation du système d'acquisition a été marquée par plusieurs défis techniques :

- ☐ **Le bus 1-Wire et la détection** : La difficulté majeure a été la mise en œuvre du double bus. Initialement, la détection des capteurs était instable sur la Raspberry Pi. J'ai dû

configurer précisément le fichier config.txt (activation des GPIO 4 et 17) pour stabiliser la reconnaissance des 6 sondes.

- ❑ **Connexion et persistance des données** : Si l'intégration des librairies MariaDB a été maîtrisée, la sauvegarde automatique n'a pu être finalisée qu'en simulation. L'absence de la base de données finale (sous la responsabilité de mes collaborateurs) a nécessité l'usage de **scripts Bash** (automatisation système) pour pallier ce manque.
- ❑ **Logistique et intégration** : Malgré une réception des petits composants dans les délais, le retard de livraison du brûleur a limité la phase d'intégration réelle de mon algorithme de régulation (gestion de la température par hystérésis).

Présentation du travail du candidat N°3 – Adam Cardoso

Introduction

Présentation de ma mission individuellement

Dans ce projet, ma mission portait principalement sur la mise en place de l'accès au système et le développement d'une partie de l'interface web utilisateur. Je me suis notamment chargé de la configuration réseau de la Raspberry Pi, du développement de la page web « Contrôler le séchage », de la page « Ajouter masse houblon » ainsi que de la mise en place d'un système d'alerte visuelle à l'aide d'un voyant lumineux.

Déroulement des tâches réalisées

1. Conception du site web « Contrôler le séchage » - Structure HTML

La première tâche que j'ai réalisé a été la création de la page « Contrôler le séchage » du site web. Dans un premier temps, avant toute intégration du langage PHP, j'ai travaillé sur le squelette de l'interface de contrôle du séchoir. Cette phase m'a permis d'expérimenter différentes dispositions possibles afin de déterminer comment organiser efficacement les éléments de la page (état du cycle, étages, variétés, dates, capteurs). Par exemple, chaque étage du séchoir est représenté par un bloc dédié dans la page HTML :

```

1 <!-- == ÉTAGE 3 == -->
2 <section class="mb-3">
3   <h2 class="section-title">
4     <span class="section-num">03</span>
5     Étage 3 – Niveau moyen
6   </h2>
7
8   <div class="floor-card">
9     <div class="row g-2">
10
11       <div class="col-12 col-md-4">
12         <div class="info-block">
13           <div class="info-label">✂ Variété</div>
14           <div class="info-value" id="variete-3"><?= htmlspecialchars($e3['variete']) ?></div>
15         </div>
16       </div>
17
18       <div class="col-12 col-md-4">
19         <div class="info-block">
20           <div class="info-label">🕒 Début</div>
21           <div class="info-value" id="debut-3"><?= htmlspecialchars($e3['date_debut']) ?></div>
22         </div>
23       </div>
24
25       <div class="col-12 col-md-4">
26         <div class="info-block">
27           <div class="info-label">🕒 Fin prévue</div>
28           <div class="info-value" id="fin-3"><?= $e3['date_fin'] ?? 'En cours' ?></div>
29         </div>
30       </div>
31
32       <div class="col-12 col-md-4 btn-block">
33         <button onclick="showOverlay(3)">Étage 3</button>
34         <button onclick="descendre(3)">↓</button>
35       </div>
36     </div>
37   </div>
38 </section>
39
40

```

Cette organisation permet une lecture claire et hiérarchisée des informations liées à chaque niveau du séchoir.

1.1 Mise en forme et design – CSS

Une fois la structure HTML finalisée, le CSS a été travaillé en collaboration avec un de mes camarades. Le design de l'interface n'étant attribué à aucun membre précis du groupe, nous avons défini ensemble la mise en forme générale (alignement des blocs, tailles de texte, hiérarchie visuelle). Cette étape a permis de rendre l'interface plus lisible et plus adaptée à une utilisation en tant qu'outil de supervision. La partie CSS est divisée en plusieurs fichiers CSS et chacun de ces fichiers s'occupe d'un aspect du design. Par exemple, « info-block » qu'on voit à la ligne 12 de la capture est définit dans le fichier « gestion-card-content.css » par

```

.info-block {
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 4px;
  padding: 0.35rem 0.6rem;
  height: 100%;
}

```

1.2 Développement PHP

Après avoir terminé la disposition de l'interface, j'ai travaillé sur la partie PHP afin de relier le site à la base de données. Le PHP permet de récupérer les informations stockées (étages actifs, variétés, dates, alertes) et de les transmettre à l'interface. Par exemple, le script `get_status.php` récupère les étages actifs ainsi que leurs informations associées grâce à une requête SQL :

```
SELECT
    houbEtag.houbEtag_etage AS etage,
    houbVar.houbVar_type AS variete,
    houbLot.houbLot_dateDebut AS date_debut,
    houbLot.houbLot_dateFin AS date_fin
FROM houbEtag
JOIN houbLot ON houbLot.id_houbLot = houbEtag.houbEtag_houbLot
JOIN houbVar ON houbVar.id_houbVar = houbLot.houbLot_houbVar
WHERE houbEtag.houbEtag_actif = 1
ORDER BY houbEtag.houbEtag_etage ASC;
```

Le PHP permet également de déterminer automatiquement l'état global du cycle de séchage :

```
$etat_cycle = count($etages) > 0 ? "Actif" : "Inactif";
```

Ces données sont ensuite envoyées au format JSON pour être exploitées côté client :

```
echo json_encode([
    "etat_cycle" => $etat_cycle,
    "derniere_alerte" => $derniere_alerte,
    "etages" => $etages
]);
```

1.3 Affichage des températures

La page permet d'avoir un suivi en temps réel des températures. Ces températures sont récupérées directement depuis les capteurs via liaison série. L'affichage de ces températures se fait sur le dernier étage car les 6 capteurs sont placés dans cet étage. Cette partie est importante car elle permet à l'agriculteur de visualiser la répartition de la température à l'intérieur du séchoir et si la plage de température idéale est respectée.

1.4 Gestion de la pause et de la reprise du séchoir

Sur cette page, on peut retrouver au début juste en-dessous de l'état deux boutons qui servent à mettre pause et reprendre le cycle de séchage. Cela permettra à l'agriculteur de mettre pause au cycle de séchage lorsqu'il voudra vérifier si son houblon est assez sec ou non. Ci-dessous, un aperçu de ce que propose la page web. Bien sûr, certaines fonctionnalités n'ont pas été faites par moi car nous avons dû travailler à deux sur cette page web.

H Séchoir Houblon

État du cycle

ÉTAT : Inconnu ⚠ DERNIÈRE ALERTE
Aucune pause

Pause Reprendre

04 Étape 4 — Niveau haut

VARIÉTÉ : — DÉBUT : — FIN PRÉVUE : —

Étape 4 ↓

03 Étape 3 — Niveau moyen

VARIÉTÉ : — DÉBUT : — FIN PRÉVUE : —

Étape 3 ↓

02 Étape 2 — Niveau bas

VARIÉTÉ : — DÉBUT : — FIN PRÉVUE : —

Étape 2 ↓

01 Étape 1 — Niveau tiroir

VARIÉTÉ : — DÉBUT : — FIN PRÉVUE : —

TEMPÉRATURE MOYENNE DES CAPTEURS : — °C

CAPTEUR 1 : — °C CAPTEUR 2 : — °C CAPTEUR 3 : — °C CAPTEUR 4 : — °C CAPTEUR 5 : — °C CAPTEUR 6 : — °C

Chargement

Les boutons Pause et Reprendre fonctionnent de la manière suivante :

Lorsque l'agriculteur appuie sur le bouton Pause, il déclenche un événement identifié par « id=btnPauseSechoir ». Au chargement de la page, le fichier GestionPAUSE.js a été exécuté. Donc dès que la page est prête, le bouton Pause est relié à sa fonction « pauseSechoir() ».

```
document.addEventListener("DOMContentLoaded", () => {
  const btnPause = document.getElementById("btnPauseSechoir");
  const btnReprendre = document.getElementById("btnReprendreSechoir");

  if (btnPause) {
    btnPause.addEventListener("click", pauseSechoir);
  }

  if (btnReprendre) {
    btnReprendre.addEventListener("click", reprendreSechoir);
  }
});
```

Le fonctionnement est le même pour le bouton Reprendre avec un point différent qu'on va voir. La fonction JavaScript envoie une requête vers le serveur. La requête est asynchrone et la page ne se recharge pas. Le serveur PHP prend le relais. La requête SQL exécutée est donc :

```
UPDATE etatSechoir
SET
  etatSechoir_status = 'pause',
  etatSechoir_pauseDebut = NOW(),
  etatSechoir_dataMaj = NOW()
WHERE id_etatSechoir = 1
AND etatSechoir_status != 'pause'
```

Cette requête passe le status du séchoir à « pause » et empêche la double pause grâce à

```
AND etatSechoir_status != 'pause'
```

Deux réponse du serveurs vers le navigateur sont possibles :

```
{  
  "status": "ok"  
}
```

Ou :

```
{  
  "status": "ignored",  
  "message": "Le séchoir est déjà en pause."  
}
```

```
const data = await res.json();  
  
if (data.status === "ok") {  
  console.log("Séchoir repris");  
  setPauseButtonsState("en_cours");  
  rafraichirStatus();  
}
```

Si la pause est acceptée, JavaScript considère que le séchoir est en pause et l'interface est mise à jour avec « rafraichirStatus() » qui est une fonction présente dans un autre fichier et qui permet de relire l'état du séchoir en base de données.

Pour reprendre l'état du séchoir, ce qui change est la requête SQL qui sera donc :

```
UPDATE etatSechoir  
SET  
  etatSechoir_ajoutMinute = etatSechoir_ajoutMinute  
    + TIMESTAMPDIF(MINUTE, etatSechoir_pauseDebut, NOW()),  
  etatSechoir_pauseDebut = NULL,  
  etatSechoir_status = 'en_cours',  
  etatSechoir_dataMaj = NOW()  
WHERE id_etatSechoir = 1  
AND etatSechoir_pauseDebut IS NOT NULL  
AND etatSechoir_status = 'pause'
```

Cette requête va calculer le temps que le séchoir a été mis en pause grâce à la fonction TIMESTAMPDIF afin d'éviter de compter le temps de pause comme du temps de séchage. Elle remet ensuite la date de début de pause à NULL et repasse l'état du séchoir à en_cours.

2. Configuration du point d'accès Wi-Fi

2.1 Objectif de la configuration

L'un des objectifs majeurs du projet est de permettre à l'agriculteur d'accéder au système de supervision du séchoir sans dépendre d'une connexion Internet. Pour répondre à ce besoin, la Raspberry Pi 5 a été configurée comme point d'accès Wi-Fi. Ce réseau permet à un smartphone de se connecter directement au séchoir et d'accéder au site web hébergé localement sur la Raspberry Pi. Cette solution présente plusieurs avantages :

- autonomie complète du système,
- simplicité d'utilisation pour l'utilisateur final,
- fiabilité accrue en environnement agricole isolé.

2.2 Installations des services nécessaires

Afin de transformer la Raspberry en point d'accès Wi-Fi, deux services principaux ont été utilisés :

- `hostapd` → permet de créer un point d'accès Wi-Fi
- `dnsmasq` → gère l'attribution automatique des adresses IP (DHCP) et la résolution DNS locale

L'installation de ces services a été effectuée à l'aide du gestionnaire de paquets :

```
1 sudo apt update
2 sudo apt install hostapd dnsmasq -y
```

Par défaut, le service `hostapd` peut-être désactivé (masqué). Il a donc été nécessaire de le débloquent et de l'activer au démarrage :

```
1 sudo systemctl unmask hostapd
2 sudo systemctl enable hostapd
3 sudo systemctl start hostapd
```

3.3 Configuration du service hostapd

Création du fichier de configuration

Le fichier de configuration principal de `hostapd` a été créé afin de définir les paramètres du réseau Wi-Fi diffusé par la Raspberry :

```
1 sudo nano /etc/hostapd/hostapd.conf
```

Configuration utilisée :

```
1 interface=wlan0
2 driver=nl80211
3 ssid=MonWiFiPi
4 hw_mode=g
5 channel=7
6
7 auth_algs=1
8 ignore_broadcast_ssid=0
9
10 wpa=2
11 wpa_passphrase=motdepasse123
12 wpa_key_mgmt=WPA-PSK
13 rsn_pairwise=CCMP
```

Cette configuration permet :

- la diffusion du réseau Wi-Fi « MonWifiPi »,
- une sécurisation via le protocole WPA2,
- une compatibilité avec les téléphones et appareils mobiles.

Déclaration du fichier de configuration

Le chemin du fichier de configuration a ensuite été associé au service hostapd :

```
1 sudo nano /etc/default/hostapd
```

Ajout de la ligne suivante :

```
1 DAEMON_CONF="/etc/hostapd/hostapd.conf"
```

3.4 Attribution d'une adresse IP statique

Pour assurer la stabilité du réseau et permettre la résolution DNS locale, une adresse IP statique a été attribuée à l'interface Wi-Fi wlan0 :

```
1 sudo ip addr add 192.168.1.254/24 dev wlan0
2 sudo ip link set wlan0 up
```

Cette adresse correspond à la passerelle du réseau local Wi-Fi utilisé par le hotspot.

3.5 Configuration du service dnsmasq

Sauvegarde de l'ancienne configuration

Avant toute modification, le fichier de configuration initial a été sauvegardé :

```
1 sudo mv /etc/dnsmasq.conf /etc/dnsmasq.conf.backup
```

Création de la nouvelle configuration

Un nouveau fichier a ensuite été créé :

```
1 sudo nano /etc/dnsmasq.conf
```

Configuration appliquée :

```
interface=wlan0
dhcp-range=192.168.1.2,192.168.1.20,255.255.255.0,24h

# Nom local pour accéder à la Raspberry Pi
address=/monpi.local/192.168.1.254
```

Cette configuration permet :

- l'attribution automatique d'adresses IP aux appareils connectés,
- l'accès au serveur web via le nom monpi.local, plus simple à retenir qu'une adresse IP.

Le service a ensuite été redémarré et activé au démarrage :

```
1 sudo systemctl restart dnsmasq
2 sudo systemctl enable dnsmasq
```


3.6 Gestion de NetworkManager

Sur les versions récentes de Raspberry Pi OS, NetworkManager peut interférer avec le fonctionnement de hostapd. Il a donc été nécessaire de déclarer l'interface wlan0 comme non gérée :

```
1 sudo nano /etc/NetworkManager/NetworkManager.conf
```

Ajout de la configuration suivante :

```
1 [keyfile]
2 unmanaged-devices=interface-name:wlan0
```

Puis redémarrage du service :

```
1 sudo systemctl restart NetworkManager
```

3.7 Démarrage du point d'accès

Une fois l'ensemble des services configurés, le hotspot a été démarré :

```
1 sudo systemctl restart hostapd
2 sudo systemctl restart dnsmasq
```

Les statuts ont été vérifiés afin de s'assurer du bon fonctionnement :

```
1 sudo systemctl status hostapd
2 sudo systemctl status dnsmasq
```

3.8 Accès au serveur web depuis un smartphone

Après connexion au réseau MonWifiPi, le téléphone reçoit automatiquement une adresse IP. Le serveur web local est accessible depuis un navigateur via l'adresse <http://monpi.local> . Dans le cas d'un serveur PHP intégré :

Php -S 0.0.0.0:8080

L'accès se fait alors via :

<http://monpi.local:8080>

3.9 Problèmes rencontrés et solutions apportées

Plusieurs difficultés ont été rencontrées lors de la configuration, notamment :

- hostapd masqué par défaut,
- conflits avec NetworkManager,
- absence d'attribution d'adresse IP aux clients Wi-Fi.

Ces problèmes ont été résolus par :

- le déblocage manuel du service hostapd,
- la déclaration de wlan0 comme interface non gérée,
- la vérification de la cohérence entre l'IP statique et la plage DHCP.

3.91 Résultat final

A l'issue de cette configuration :

- la Raspberry Pi 5 diffuse automatiquement le réseau Wi-Fi « MonWifiPi »
- les appareils connectés obtiennent une adresse IP via dnsmasq,
- le site web du séchoir est accessible via <http://monpi.local>,
- le hotspot démarre automatiquement au démarrage de la Raspberry.

Cette solution répond pleinement au besoin d'autonomie et de simplicité d'accès du projet.

4. Conception de la page « Ajouter masse houblon »

4.1 Objectif de la page

La page « Ajouter masse houblon » a été développée afin de permettre à l'agriculteur d'enregistrer la masse finale de houblon après séchage.

En effet, une fois le cycle de séchage terminé, l'agriculteur pèse la production et souhaite :

- Associer une masse finale à un ou plusieurs lots,
- Conserver une trace de cette masse en base de données,
- Centraliser ces informations pour un suivi ultérieur.

Cette page constitue donc une étape importante dans le cycle de production, permettant de passer d'un suivi du séchage à un suivi des résultats.



4.2 Architecture générale de la page

La page repose sur une architecture classique client / serveur :

- Front-End (côté client) :
 - Interface HTML générée par PHP
 - Interactions dynamiques en JavaScript (ajout_masse.js)
- Back-End (côté serveur) :
 - Traitement des données en PHP
 - Interactions avec la base de données MySQL via PDO

Cette séparation permet :

- Une interface réactive côté utilisateur,
- Un traitement sécurisé côté serveur,
- Une maintenance facilitée.

4.3 Interface utilisateur

L'interface est organisée en deux parties principales :

Sélection des lots

La partie gauche de l'interface affiche la liste des lots disponibles. Ces lots sont récupérés dynamiquement depuis la base de données via une requête SQL :

```
SELECT lot.id_lot, variete.variete_nom
FROM lot
INNER JOIN variete ON lot.id_variete = variete.id_variete
WHERE lot.id_masse IS NULL
AND lot.lot_dateHeureSortie IS NOT NULL
```

Cette requête permet de :

- Afficher uniquement les lots ayant terminé leur séchage,
- Exclure ceux déjà associés à une masse.

L'utilisateur peut sélectionner un ou plusieurs lots via des cases à cocher.

Saisie de la masse

Sur la partie droite, un champ de saisie permet à l'utilisateur d'entrer la masse totale en kilogrammes.

Des contrôles sont appliqués directement en Javascript afin de garantir la validité des données :

- Autorisation uniquement des nombres,
- Limitations à 2 chiffres après la virgule.

Exemple :

```
if (!/^\d*\.\?\d{0,2}$/.test(valeur))
```

Cela permet d'éviter des erreurs de saisie dès l'interface utilisateur.

Interaction et validation des données

Lors du clic sur le bouton « Ajouter la masse », plusieurs vérifications sont effectuées côté client :

- Au moins un lot doit être sélectionné,
- La masse doit être valide (format numérique avec 2 décimales maximum).

En cas d'erreur, un message est affiché à l'utilisateur sous forme d'alerte.

Système de confirmation

Avant l'envoi des données, une fenêtre de confirmation (overlay) est affichée. Cette fenêtre permet à l'utilisateur de :

- Vérifier les lots sélectionnés,
- Vérifier la masse saisie,
- Confirmer ou annuler l'opération.

Cette étape améliore la fiabilité du système en évitant les erreurs de validation accidentelles.

Envoi des données au serveur

Une fois confirmées, les données sont envoyées au serveur via une requête AJAX utilisant l'API Fetch :

```
fetch("/site/ajout_masse.php", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    masse: masseSelectionnee,
    id_lots: lotsSelectionnees.map(lot => lot.id)
  })
})
```

Cette approche présente plusieurs avantages :

- Pas de rechargement de la page,
- Meilleure expérience utilisateur,
- Communication structurée en JSON

4.4 Traitement côté serveur (PHP)

Le fichier PHP reçoit les données envoyées par le client et les traite en plusieurs étapes.

Validation des données

Le serveur vérifie :

- Que la masse est bien définie et respecte le format attendu,
- Que les identifiants des lots sont valides.

Exemple :

```
if ($masse === null || !preg_match('/^\d+(\.\d{1,2})?$/', $masse))
```

Cette validation côté serveur est essentielle pour éviter :

- Les erreurs de saisies,
- Les injections malveillantes.

4.5 Insertion en base de données

Le traitement est réalisé dans une transaction SQL, garantissant la cohérence des données.

Étapes réalisées :

1. Insertion de la masse :

```
INSERT INTO masse (masse_masse, masse_dateHeure)
VALUES (:masse, NOW())
```

2. Récupération de l'identifiant de la masse et association de cette masse au lots sélectionnés :

```
UPDATE lot
SET id_masse = ?
WHERE id_lot IN (...)
```

La transaction permet de :

- Garantir que toutes les opérations réussissent ou échouent ensemble,
- Éviter les incohérences en base de données.

4.6 Gestion des réponses serveur

Le serveur renvoie une réponse au format JSON :

```
{
  "success": true,
  "message": "Masse ajoutée avec succès."
}
```

Côté JavaScript :

- La réponse est analysée,

- Un message est affiché à l'utilisateur,
- L'interface est réinitialisée en cas de succès.

4.7 Réinitialisation de l'interface

Après un enregistrement réussi :

- Le champ de saisie est vidé,
- Les lots sont désélectionnés,
- La fenêtre de confirmation est fermée.

Cela permet à l'utilisateur de réaliser une nouvelle saisie rapidement.

4.8 Bilan de la conception

La page « Ajouter masse houblon » permet de répondre efficacement au besoin de suivi de production de l'agriculteur.

Elle offre :

- Une interface claire et intuitive,
- Une validation robuste des données,
- Une interaction fluide avec la base de données,
- Une bonne séparation entre interface et traitement.

5. Choix et mise en place du voyant lumineux

Une alerte visuelle doit être mise en place afin d'alerter l'agriculteur de la fin d'un cycle de séchage.

Le voyant lumineux doit respecter les critères suivants selon le cahier des charges :

- Visibilité suffisante dans un environnement lumineux
- Compatibilité avec la Raspberry Pi 5 fournies
- Fiabilité sur de longues durées de fonctionnement
- Consommation électrique faible.

5.1 Choix du voyant

J'ai repéré trois choix lors de ma recherche :

Nom	Voyant LED 5V - VE5VCR	ELMARK Voyant LED Rouge 24V AC/DC - EM401424	Voyant monobloc à LED IMO 24VAC/DC - LMB-24-WHI
Type	LED simple 5V	Voyant industriel 24V	Voyant industriel Ø22 mm
Prix	2,93€	2,99€	7,07€
Compatibilité RaspBerry Pi	Directe via GPIO	Nécessite transistor/relais	Nécessite transistor/relais
Facilité d'installation	Très simple	Moyenne	Moyenne
Intensité lumineuse	30mcd	1000-2000mcd	2000-3000mcd
Robustesse	Standard	Industrielle	Industrielle
Consommation	Environ 13mA	Environ 15-20mA	Environ 20mA
Adapté environnement agricole	Moyen	Oui	Oui
Résistance vibrations/poussières	Moyenne	Bonne	Bonne
Durée de vie estimée	50 000h	50 000h	50 000h

mcd = millicandela

Choix final :

ELMARK Voyant LED Rouge 24V AC/DC - EM401424

Car :

- Seulement 2,99€
- Beaucoup plus visible que la LED 5V
- Adapté milieu agricole
- coût très faible par rapport aux performances

Cependant, le lycée avait déjà des voyants lumineux en stock donc je vais utiliser pour ce projet celui fourni par le lycée.